

UNIVERSIDAD DE MÁLAGA

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA DE
TELECOMUNICACIÓN



TRABAJO FIN DE MÁSTER

VICON: Sistema de Visión configurable

Máster en Sistemas Electrónicos
para Entornos Inteligentes

MÁLAGA, 2026

Carlos Manuel Gómez Jiménez

VICON: Sistema de Visión configurable

REALIZADO POR:

Carlos Manuel Gómez Jiménez

DIRIGIDO POR:

Martín González García

DEPARTAMENTO:

Departamento de Tecnología Electrónica

TITULACIÓN:

Máster en Sistemas Electrónicos para Entornos Inteligentes

PALABRAS CLAVE:

FPGA, VHDL, RTL, Síntesis Lógica, sensor CMOS, visión artificial, OpenCV.

RESUMEN

Este trabajo presenta el diseño, implementación y verificación de VICON, un sistema de adquisición y transmisión de imagen en tiempo real basado en FPGA. El sistema integra el sensor de imagen CMOS MT9V111 con la plataforma Basys 3 (Xilinx Artix-7) y transmite el flujo de vídeo hacia un PC mediante el chip FT232H de FTDI operando en modo *Synchronous FIFO* sobre un enlace de 60 MHz. El diseño hardware, desarrollado íntegramente en VHDL, comprende un controlador I2C para la configuración del sensor, un módulo de captura de trama con gestión de marcadores de protocolo, un controlador de alta velocidad para el FT232H con arquitectura de doble proceso para TX y RX, y un procesador de comandos con sincronización entre dominios de reloj (CDC). La verificación se realiza mediante simulación y con pruebas en Hardware, en la primera empleando agentes para el sensor I2C y para el chip FT232H. El software de visualización, desarrollado en C++ con Qt y OpenCV, recibe el flujo de imagen y permite enviar comandos de control a la FPGA. El sistema completo ha sido validado sobre hardware real, demostrando la transmisión de imagen en escala de grises a resolución VGA (640×480) y la correcta ejecución de todos los comandos de control.

ABSTRACT

This work presents the design, implementation and verification of VICON, a real-time image acquisition and transmission system based on FPGA. The system integrates the CMOS image sensor MT9V111 with the Basys 3 platform (Xilinx Artix-7) and transmits the video stream to a PC through the FTDI FT232H chip, operating in *Synchronous FIFO* mode over a 60 MHz link. The hardware design, developed entirely in VHDL, comprises an I2C controller for sensor configuration, a frame capture module with protocol marker handling, a high-speed controller for the FT232H featuring a dual-process architecture for TX and RX, and a command processor with clock domain crossing (CDC) synchronisation. Verification is carried out through simulation and hardware testing; the former employs UVVM-based agents for both the I2C sensor and the FT232H chip. The visualisation software, developed in C++ with Qt and OpenCV, receives the image stream and allows control commands to be sent to the FPGA. The complete system has been validated on real hardware, demonstrating the transmission of grayscale images at VGA resolution (640×480) and the correct execution of all control commands.

AGRADECIMIENTOS

Quiero agradecer en primer lugar a mi tutor, por su disponibilidad, sus comentarios y la orientación prestada durante el desarrollo de este trabajo.

A mi familia y a Desi, por su apoyo y paciencia a lo largo de todos los altibajos de este proceso.

ÍNDICE GENERAL

1	INTRODUCCIÓN	1
1.1	Motivación	1
1.2	Objetivos	1
1.3	Entorno de usuarios	2
1.4	Directivas del proyecto	2
1.5	Alternativas y competencia	3
1.6	Planificación temporal	4
1.7	Estructura de la memoria	4
2	ESTADO DEL ARTE	7
2.1	Sistemas de visión artificial embebida	7
2.2	Interfaces de alta velocidad hacia PC	7
2.3	Sensores de imagen CMOS	8
2.4	Metodologías de verificación de hardware digital	8
3	HERRAMIENTAS Y ENTORNO DE DESARROLLO	9
3.1	Diseño e implementación en FPGA	9
3.2	Simulación y verificación	9
3.3	Control de versiones	10
3.4	Documentación	10
4	REQUISITOS Y CASOS DE USO	11
4.1	Requisitos del sistema	11
4.1.1	Descripción de los requisitos funcionales	12
4.1.2	Descripción de los requisitos no funcionales	13
4.2	Casos de uso	14
4.2.1	Descripción textual de los casos de uso	14
5	DESCRIPCIÓN Y ARQUITECTURA DEL SISTEMA	19
5.1	Arquitectura global	19
5.2	Arquitectura física	20
5.3	Hardware	20
5.3.1	FPGA Basys 3	20
5.3.2	Sensor CMOS MT9V111	21
5.3.3	Chip FT232H	23
5.4	Arquitectura lógica	26
5.4.1	Componentes de la FPGA	27
5.4.2	Relaciones entre componentes y dominios de reloj	28
5.4.3	Componentes del Host	30
6	DESARROLLO	31
6.1	Estructura del proyecto	31
6.2	RTL Design	32
6.2.1	TOP	32
6.2.2	I2C Controller	33
6.2.3	Frame Capture	36

6.2.4	FTDI Controller	37
6.2.5	CMD Processor	38
6.2.6	Imagen sintética (cam_sim)	39
6.3	Gestión de cruces de dominio de reloj (CDC)	39
6.4	Verificación por Simulación	40
6.4.1	Estructura del banco de pruebas	40
6.4.2	Agente FTDI	41
6.4.3	Monitores	41
6.4.4	Resultados de simulación	42
7	SOFTWARE DE VISUALIZACIÓN PC	45
7.1	Arquitectura y concurrencia	45
7.2	Interfaz gráfica	46
7.3	Protocolo de recepción de imagen	47
7.4	Protocolo de comandos y control	47
7.5	Detección de rostros	48
7.6	Gestión de errores y limitaciones	48
8	IMPLEMENTACIÓN Y RESULTADOS	51
8.1	Síntesis e implementación en Vivado	51
8.1.1	Resultados de timing	51
8.1.2	Utilización de recursos	52
8.2	Constraints XDC	53
8.3	Depuración en hardware con ILA	53
8.3.1	Diagnóstico del protocolo FT232H	54
8.4	Pruebas en hardware	54
8.4.1	Transmisión de imagen	54
8.4.2	Comandos de control	54
9	VERIFICACIÓN Y VALIDACIÓN	57
9.1	Verificación de requisitos	57
9.1.1	Entornos de prueba	57
9.1.2	Descripción de las pruebas	58
9.2	Plan de validación	64
9.2.1	Pruebas del plan de validación	65
10	CONCLUSIONES Y TRABAJO FUTURO	71
10.1	Conclusiones	71
10.2	Trabajo futuro	72
	BIBLIOGRAFÍA	73
I	APENDICES	
A	MANUAL DE USUARIO	77
A.1	Requisitos del sistema	77
A.2	Instalación y configuración	77
A.2.1	Descarga del código fuente	77
A.2.2	Configuración del módulo FT232H	78
A.2.3	Compilación de la aplicación de PC	78

A.3	Guía de uso de la aplicación	78
A.3.1	Inicio de la aplicación	78
A.3.2	Descripción de la interfaz	78
A.4	Resolución de problemas (<i>troubleshooting</i>)	81
B	DIAGRAMAS DE ESTADOS (FSM)	83
B.1	FSM de configuración del TOP	83
B.2	FSM del i2c_controller	83
B.3	FSM de recepción del ftdi_controller	83

ÍNDICE DE FIGURAS

Figura 1.1	Planificación temporal del proyecto.	4
Figura 4.1	Diagrama general de requisitos.	12
Figura 4.2	Diagrama de casos de uso.	15
Figura 5.1	Arquitectura global del sistema VICON.	19
Figura 5.2	Descomposición parcial de la arquitectura global.	20
Figura 5.3	Arquitectura física del sistema VICON (diagrama de bloques). .	20
Figura 5.4	Captura con analizador lógico de las señales del MT9V111 (RESETN, PIXLCLK, VSYNC, HREF, D[7:0]) durante la fase inicial de caracterización.	22
Figura 5.5	Conexión del sensor CMOS a la Basys 3.	24
Figura 5.6	Arquitectura lógica del sistema VICON: visión general.	27
Figura 5.7	Arquitectura lógica de la FPGA: módulos, señales y dominios de reloj.	29
Figura 6.1	Tramas para escritura y lectura de 16 bits en el bus I2C del sensor MT9V111.	35
Figura 6.2	Cronograma de las 4 fases de SCLK.	35
Figura 6.3	Diagrama de bloques del banco de pruebas de VICON.	41
Figura 6.4	Vista general de la simulación: configuración I2C, inicio de captura y ejecución de comandos.	43
Figura 6.5	Detalle del generador de imagen sintética: contadores de colum- na y fila de frame_capture.	44
Figura 6.6	Recepción y decodificación de un comando LED (CMD 0x01) durante la simulación.	44
Figura A.1	Estado Inicial de la Interfaz Gráfica de Usuario.	79
Figura A.2	GUI: envío de comando I2C para configurar >30fps.	79
Figura A.3	GUI: envío de comando I2C para configurar >30fps.	80
Figura B.1	FSM de configuración del sensor en TOP.vhd.	84
Figura B.2	FSM de i2c_controller.vhd.	85
Figura B.3	FSM de recepción (p_rx) de ftdi_controller.vhd.	86

ÍNDICE DE TABLAS

Tabla 1.1	Descripción del problema y de la solución propuesta.	3
Tabla 4.1	Requisitos del proyecto.	11
Tabla 4.2	Actores del sistema.	14
Tabla 5.1	Asignación de pines de la FPGA para el sensor MT9V111. . . .	23
Tabla 5.2	Asignación de pines de la FPGA para el bus de datos del FT232H (ADBUS).	25
Tabla 5.3	Asignación de pines de la FPGA para el bus de control del FT232H (ACBUS).	26
Tabla 6.1	Estados de la FSM de <code>i2c_controller.vhd</code>	36
Tabla 6.2	Cruces de dominio de reloj en el diseño VICON.	39
Tabla 7.1	Comandos PC→FPGA definidos en <code>mainwindow.cpp</code>	47
Tabla 8.1	Resumen de <i>timing</i> tras la implementación.	51
Tabla 8.2	Desglose del peor camino (<code>s_wr_n_reg</code> → <code>WR#</code>).	52
Tabla 8.3	Utilización de recursos en la XC7A35T.	53
Tabla 9.1	Prueba P1.	58
Tabla 9.2	Prueba P2.	59
Tabla 9.3	Prueba P3.	60
Tabla 9.4	Prueba P4.	61
Tabla 9.5	Prueba P5.	62
Tabla 9.6	Prueba P6.	62
Tabla 9.7	Prueba P7.	63
Tabla 9.8	Prueba P8.	63
Tabla 9.9	Prueba P9.	64
Tabla 9.10	Entregables y pruebas de validación asociadas.	64
Tabla 9.11	Prueba de validación V1.	65
Tabla 9.12	Prueba de validación V2.	66
Tabla 9.13	Prueba de validación V3.	67
Tabla 9.14	Prueba de validación V4.	68
Tabla 9.15	Prueba de validación V5.	69
Tabla A.1	Comandos disponibles desde la interfaz de <code>vicon_view</code>	80
Tabla A.2	Problemas frecuentes y soluciones.	81

ACRONIMOS

PCB	Printed Circuit Board
VGA	Video Graphics Array
FPGA	Field-Programmable Gate Array
VHDL	VHSIC Hardware Description Language
CDC	Clock Domain Crossing
UVVM	Universal VHDL Verification Methodology
TFM	Trabajo Fin de Máster
VICON	Sistema de Visión Configurable
RTL	Register Transfer Level
FIFO	First In - First Out
SW	Software
HW	Hardware
TB	TestBench
VVC	VHDL Verification Component
UVM	Universal Verification Methodology
OSVVM	Open Source VHDL Verification Methodology
ILA	Integrated Logic Analyzer
IP	Intellectual Property
IDE	Entorno de Desarrollo Integrado
IFP	Image Flow Processor
FPS	Frames por segundo
SCL	Serial Clock
SDA	Serial Data
FSM	Finite State Machine
MMCM	Mixed-Mode Clock Manager

GUI	Graphical User Interface
FWFT	First Word Fall Through
I²C	Inter-Integrated Circuit
DUT	Device Under Test
CCIO	Clock Capable Input/Output
LUT	Look Up Table
IOB	Input/Output Block
PS	Processing System
ACK	Acknowledge
EEPROM	Electrically Erasable Programmable Read-Only Memory
SBC	Single Board Computer
PLL	Phase-locked loop

INTRODUCCIÓN

Este capítulo introduce el proyecto **VICON** (Sistema de Visión Configurable), presentando la motivación que lo origina, los objetivos perseguidos, el entorno de usuarios al que se dirige, las directivas seguidas durante su desarrollo, las alternativas existentes y la planificación temporal, y cerrando con la estructura seguida en el resto de la memoria.

1.1 MOTIVACIÓN

La visión artificial constituye uno de los campos de mayor crecimiento dentro de la ingeniería electrónica actual. La capacidad de adquirir, procesar y transmitir imágenes en tiempo real desde plataformas de bajo coste y reducidas dimensiones abre la puerta a multitud de aplicaciones industriales, médicas y de seguridad. Sin embargo, el diseño de estos sistemas presenta retos técnicos significativos: el ancho de banda necesario para transportar estos flujos de vídeo sin comprimir, la latencia introducida por las interfaces de comunicación y la sincronización entre dominios de reloj diferentes son problemas que requieren soluciones a nivel de Hardware.

Las **FPGA** (Field-Programmable Gate Array)s ofrecen ventajas únicas frente a los microcontroladores o los procesadores de propósito general: paralelismo, la posibilidad de implementar la lógica de captura y transmisión de imagen sin necesidad de sistemas operativos ni de *drivers* complejos, etc.

La Basys 3, basada en la familia Artix-7 de Xilinx, es una plataforma accesible y utilizada en entornos académicos que permite demostrar un sistema de visión embebido completo.

1.2 OBJETIVOS

El presente **TFM** (Trabajo Fin de Máster) tiene como objetivo principal el diseño, implementación y verificación de **VICON**, un sistema de adquisición (captura) y transmisión de imagen configurable basado en **FPGA**. Los objetivos específicos son:

- Diseñar en **VHDL** (VHSIC Hardware Description Language) el Hardware necesario para capturar imágenes del sensor CMOS MT9V111, incluyendo la interfaz de configuración I2C y el módulo de captura de trama.
- Implementar un controlador de alta velocidad para el chip FT232H de FTDI en modo *Synchronous FIFO* (First In - First Out), capaz de transmitir el flujo de imagen hacia un PC (Host) y recibir los comandos en sentido contrario.

- Gestionar correctamente los [CDC](#) (Clock Domain Crossing) entre los distintos dominios presentes en el diseño (reloj del sistema, reloj de píxel de MT9V111 y reloj del FT232H).
- Verificar el diseño mediante simulación, incluyendo agentes pertenecientes a [UVVM](#) (Universal VHDL Verification Methodology), así como agentes de simulación para el sensor I2C y el FT232H.
- Desarrollar una aplicación de visualización en PC o [SW](#) (Software) con Qt y OpenCV que reciba el flujo de imagen y permita enviar comandos de control a la [FPGA](#).
- Demostrar el funcionamiento del sistema completo en Hardware real sobre la plataforma Basys 3.

1.3 ENTORNO DE USUARIOS

Este [TFM](#) va dirigido principalmente a estudiantes e ingenieros con conocimientos básicos de electrónica digital y [VHDL](#) que deseen iniciarse en el diseño de sistemas de visión embebidos sobre [FPGA](#). También resulta de interés para desarrolladores que trabajen con el chip FT232H en modo *Synchronous FIFO*, dada la escasez de ejemplos completos y documentados de este modo de operación en [VHDL](#).

1.4 DIRECTIVAS DEL PROYECTO

La Tabla [1.1](#) resume, de forma esquemática, el problema abordado por este [TFM](#) y la solución propuesta.

Tabla 1.1: Descripción del problema y de la solución propuesta.

Problema	Descripción
Sistema afectado	Necesidad de un sistema de adquisición y transmisión de vídeo en tiempo real desde un sensor CMOS hacia un PC, sin recurrir a plataformas con procesador embebido (SoC/Zynq) ni a interfaces de vídeo estándar como HDMI.
Consecuencias	Los prototipos académicos existentes suelen depender de plataformas caras (Zynq) o de interfaces de alto nivel que ocultan el diseño RTL de la captura y el enlace de comunicaciones, dificultando su uso como plataforma de aprendizaje.
Solución propuesta	Descripción
Enfoque	Diseñar íntegramente en VHDL, sobre una FPGA de bajo coste (Basys 3), la cadena completa de adquisición de frames (sensor MT9V111), transmisión (FT232H en modo <i>Synchronous FIFO</i>) y visualización/control (aplicación C++/Qt/OpenCV en el Host).
Beneficios	Plataforma completa, documentada y verificada (simulación + hardware real) que sirve como demostrador funcional.

1.5 ALTERNATIVAS Y COMPETENCIA

Entre las alternativas existentes para resolver un problema similar destacan:

- **Plataformas SoC (Xilinx Zynq y similares):** combinan un procesador ARM con lógica programable, lo que simplifica notablemente el procesamiento de imagen (Linux, OpenCV nativo, *drivers* de cámara ya existentes) a costa de un mayor coste económico y de una menor transparencia sobre el diseño RTL de bajo nivel (véase la Sección 2). Se puede tomar como ejemplo el trabajo realizado en la asignatura Intensificación de FPGAs cursada en este máster.
- **Cámaras industriales inteligentes (*smart cameras*):** soluciones comerciales que integran sensor, procesado y salida de vídeo estandarizada, pensadas para producción y no para el aprendizaje del diseño de la cadena de adquisición.
- **Raspberry Pi u otras SBC (Single Board Computer) con módulo de cámara:** solución de bajo coste y fácil programación en Python/C++, pero sin control a nivel de ciclo de reloj sobre la captura ni posibilidad de implementar lógica de procesamiento en paralelo.

Frente a estas alternativas, **VICON** se diferencia por implementar toda la cadena de transmisión de datos (captura, *framing* y transmisión de alta velocidad) directamente en **RTL** (Register Transfer Level) sobre una FPGA de gama básica.

1.6 PLANIFICACIÓN TEMPORAL

El desarrollo de este **TFM** se ha organizado en fases sucesivas que abarcan desde el estudio previo hasta la redacción de la memoria. La Figura 1.1 muestra la planificación temporal mediante un diagrama de Gantt, comparando el **baseline** inicial (planificación optimista establecida al comienzo del proyecto) con la ejecución real. Las fases de diseño, desarrollo y verificación se solapan parcialmente, mostrando el carácter iterativo del trabajo: la depuración en Hardware realimentó tanto el diseño **RTL** como el **SW** de visualización.

Como ocurre habitualmente en proyectos de esta naturaleza, la ejecución real se desvió de la planificación inicial: tras un primer arranque en marzo, el proyecto sufrió una pausa relevante durante el mes de abril debida a bloqueos y agentes externos ajenos al propio desarrollo técnico (disponibilidad de tiempo, etc.). El trabajo se retomó con intensidad en los tres meses siguientes, concentrando en ese periodo final la mayor parte del desarrollo **RTL**, la validación en hardware, el desarrollo del software de visualización y la redacción de la memoria, tal como refleja el hueco visible entre los tramos de las barras reales del diagrama.

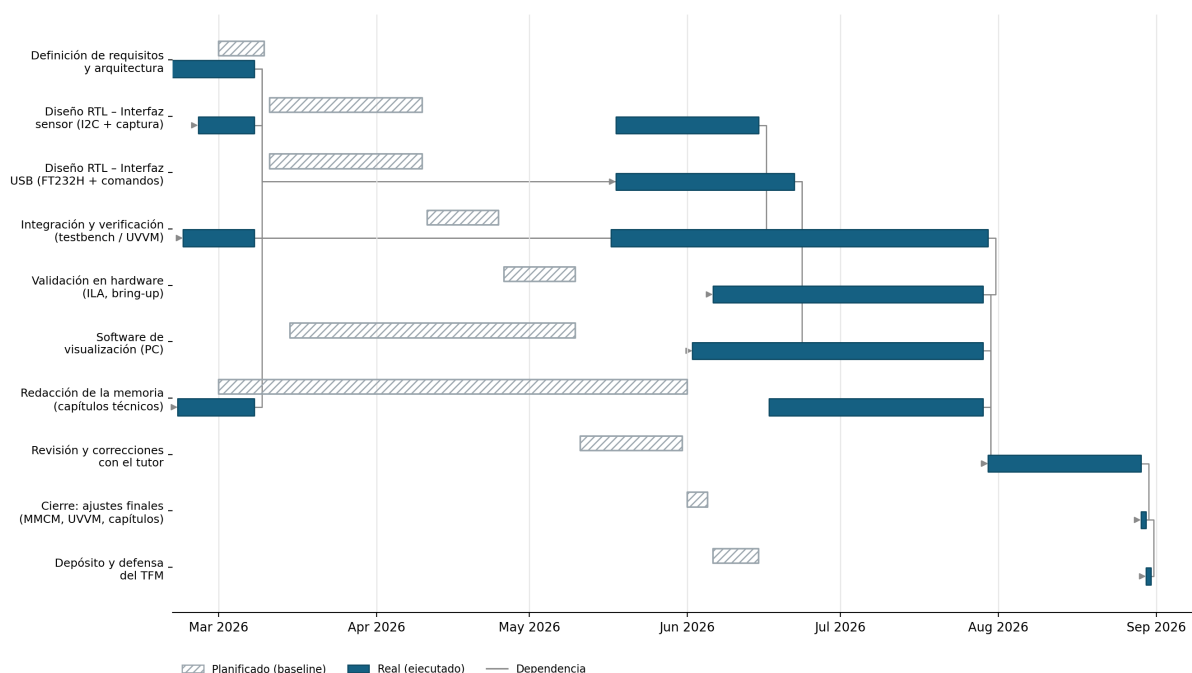


Figura 1.1: Planificación temporal del proyecto.

1.7 ESTRUCTURA DE LA MEMORIA

El presente documento se organiza de la siguiente forma: El capítulo 2 menciona brevemente el estado del arte en sistemas de visión embebida sobre FPGA, las interfaces

de alta velocidad hacia PC y las metodologías de verificación de Hardware. El capítulo 3 describe las herramientas y el entorno de desarrollo empleados. El capítulo 4 recoge los requisitos que guían el proyecto y los casos de uso a los que da respuesta. El capítulo 5 describe la arquitectura global, física y lógica del Sistema VICON, así como el Hardware utilizado. El capítulo 6 detalla el diseño a nivel RTL de cada módulo, la estrategia de CDC y la verificación mediante UVVM. El capítulo 7 describe el SW de visualización desarrollado en PC. El capítulo 8 presenta los resultados de síntesis e implementación en Vivado, el análisis de timing y las pruebas sobre Hardware real. El capítulo 9 recoge la verificación de los requisitos y el plan de validación del sistema. Finalmente, el capítulo 10 presenta las conclusiones del trabajo y las líneas de trabajo futuro.

ESTADO DEL ARTE

Este capítulo repasa el estado del arte de las principales tecnologías involucradas en el proyecto: los sistemas de visión artificial embebida, las interfaces de alta velocidad empleadas para la transmisión de vídeo hacia un PC, los sensores de imagen CMOS y las metodologías de verificación de hardware digital.

2.1 SISTEMAS DE VISIÓN ARTIFICIAL EMBEBIDA

Los sistemas de visión artificial embebida han evolucionado notablemente en la última década. Las soluciones basadas en microcontroladores, permiten capturar y procesar imágenes de baja resolución con latencias aceptables. Sin embargo, cuando se requiere el procesamiento de flujos de vídeo en tiempo real a resoluciones **VGA** (Video Graphics Array) o superiores, las **FPGAs** se presentan como la alternativa más adecuada gracias a su capacidad de procesar datos en paralelo y al acceso directo a los pines de entrada/salida sin la mediación de un sistema operativo.

Entre los trabajos más representativos en este ámbito destaca el uso de plataformas Zynq de Xilinx, que combinan un procesador ARM con lógica programable y permiten implementar *pipelines* de procesamiento de imagen con aceleración hardware. No obstante, para aplicaciones donde el objetivo principal es la adquisición y transmisión de imagen sin procesamiento local intensivo, una **FPGA** pura sin parte de **PS** (Processing System) como la Artix-7 ofrece menores costes y una complejidad de diseño más controlable.

2.2 INTERFACES DE ALTA VELOCIDAD HACIA PC

La transmisión de flujos de imagen sin comprimir hacia un PC exige interfaces de alta velocidad. Las alternativas más comunes son USB 3.0, GigE Vision y PCIe. En el presente proyecto se utiliza, por requisito, la familia FT232H de FTDI (Módulo UM232H-B). Este chip ofrece un modo *Synchronous FIFO* de 8 bits que permite alcanzar tasas de transferencia de hasta 60 MB/s sobre USB 2.0 [1], con una interfaz síncrona válida para su implementación en **VHDL**.

El modo *Synchronous FIFO* del FT232H opera con un reloj de 60 MHz generado por el propio chip. La FPGA actúa como maestro del bus: para transmitir datos (FPGA→PC) activa una señal y presenta el dato en el bus de datos; para recibir datos (PC→FPGA) activa otra señal diferente para que el FT232H controle el bus y a continuación aserta otra señal para consumir los bytes disponibles.

2.3 SENSORES DE IMAGEN CMOS

El sensor MT9V111 de ON Semiconductor es un sensor CMOS [VGA](#) (640×480) que entrega datos en formato YCbCr 4:2:2. Dispone de una **interfaz** de configuración **I2C** de dos hilos y una interfaz de **datos** paralela de 8 bits con señales de sincronismo de trama (FVALID), línea (LVALID) y reloj de píxel (PIXCLK) [\[2\]](#).

2.4 METODOLOGÍAS DE VERIFICACIÓN DE HARDWARE DIGITAL

La verificación funcional ha evolucionado desde los bancos de pruebas *ad hoc* hacia metodologías estructuradas que promueven sobre todo la reutilización y la automatización. En el ámbito de SystemVerilog, la referencia en la industria es [UVM](#) (Universal Verification Methodology) [\[3\]](#), que ofrece un entorno de verificación orientado a objetos con generación de estímulos aleatorios restringidos y cobertura funcional. Sin embargo, requiere un simulador con soporte completo de SystemVerilog, lo que la aleja de los flujos basados en VHDL.

Dentro del ecosistema VHDL han surgido varias alternativas. [OSVVM](#) (Open Source VHDL Verification Methodology) [\[4\]](#) aporta *constrained random*, cobertura funcional y modelos de *scoreboard* sobre VHDL puro. [cocotb](#) [\[5\]](#), por su parte, permite escribir los bancos de pruebas en Python, separándolo del lenguaje de descripción de hardware. Frente a estas, [UVVM](#) [\[6\]](#) propone un enfoque estructurado y modular especialmente orientado a la verificación de interfaces: proporciona una biblioteca de utilidades (`uvvm_util`) con funciones de *logging*, gestión de alertas y comprobación de valores (*checks*), y un framework de [VVC](#) (VHDL Verification Component)s que abstrae cada agente de verificación y permite coordinarlos desde un secuenciador central.

Para este trabajo se ha optado por [UVVM](#) por ser una metodología nativa de VHDL —el lenguaje en el que se describe todo el diseño—, lo que evita introducir SystemVerilog en el flujo y mantener así un único lenguaje en [RTL](#) y el banco de pruebas. Su modelo de [VVCs](#) encaja a priori de forma natural con la verificación de las interfaces del sistema, y su biblioteca de comprobaciones y alertas facilita la trazabilidad frente a los requisitos especificados.

HERRAMIENTAS Y ENTORNO DE DESARROLLO

Este capítulo recoge las herramientas y bibliotecas empleadas a lo largo del proyecto, agrupadas según la fase en la que intervienen: diseño e implementación en FPGA, simulación y verificación, control de versiones, desarrollo del software de PC y redacción de la memoria.

3.1 DISEÑO E IMPLEMENTACIÓN EN FPGA

El flujo de diseño hardware se realiza íntegramente con **Vivado 2020.2** de Xilinx. Esta herramienta cubre todas las etapas del flujo: síntesis, implementación, análisis de timing, generación de bitstream y depuración en hardware mediante el **ILA** (Integrated Logic Analyzer). El uso de **IP** (Intellectual Property)s permite reutilizar bloques funcionales ya diseñados, verificados y optimizados por el fabricante para la arquitectura deseada, lo que reduce el tiempo de desarrollo y el riesgo de errores si lo comparamos a una implementación propia con la misma funcionalidad.

3.2 SIMULACIÓN Y VERIFICACIÓN

La simulación funcional se realiza con **ModelSim/QuestaSim** de Mentor Graphics, invocado mediante scripts `.do` que automatizan la compilación, optimización y ejecución del banco de pruebas. El script principal mapea las librerías de simulación de Vivado (compiladas previamente) y las librerías **UVVM**, y lanza la simulación con resolución temporal de 1 ps.

La metodología de verificación empleada es **UVVM**, en su versión completa (no la versión Light). Las librerías utilizadas son:

- `uvvm_util`: utilidades de logging, gestión de alertas y funciones de comprobación (`check_value`, `log`, `alert`).
- `uvvm_vvc_framework`: framework de *Verification Components*.

Las librerías **UVVM** se precompilan una única vez desde el directorio raíz de la instalación (En este caso `C:/UVVM`) ejecutando `compile_all.do`, y se referencian en cada simulación mediante `vmap` en el script `sim.do`. Es posible realizar todos los pasos que se detallan en el script mediante acciones manuales en la interfaz de usuario de Questa, pero esta metodología es más propensa a errores y no permite un buen trabajo en equipos multidisciplinares.

3.3 CONTROL DE VERSIONES

El código fuente del proyecto (RTL, banco de pruebas, scripts y documentación) se gestiona con **Git**, con repositorio remoto alojado en **GitHub** (https://github.com/carlosmgj/VICON_public). La estrategia de ramas sigue el modelo *feature branching*: cada nueva funcionalidad o corrección se desarrolla en una rama independiente (feature/<nombre>) y se integra en main mediante *pull requests* una vez validada. Para este proyecto, dado que está desarrollado por una única persona, no se utilizan pull requests para mergear los cambios a la rama main, pero es lo que se considera más recomendable en entornos con múltiples desarrolladores.

Además, se ha configurado un runner de GitHub Actions con el fin de mantener una documentación actualizada del proyecto. Este runner publica la página de documentación en: https://carlosmgj.github.io/VICON_public/

El software de visualización, nombrado `vicon_view`, se desarrolla con las siguientes herramientas y bibliotecas:

- **Qt Creator** como **IDE** (Entorno de Desarrollo Integrado), con **Qt 6** para la interfaz gráfica.
- **OpenCV 4.5.5** para el procesamiento y conversión de los frames de imagen recibidos.
- **FTD2XX API** (biblioteca oficial de FTDI) para la comunicación con el chip FT232H a través de USB.
- **MinGW/GCC** como compilador C++ bajo Windows.

La primera versión de esta aplicación se desarrolló en Python, pero se abandonó por problemas de latencia de lectura que provocaban el desbordamiento de la **FIFO** asíncrona de la FPGA; la justificación detallada de este cambio de lenguaje se recoge en el Capítulo 7.

3.4 DOCUMENTACIÓN

La presente memoria se redacta en **L^AT_EX** con la distribución **MiKTeX 26.2**. Los diagramas de bloques, diagramas de estados y figuras de arquitectura se crean con **draw.io**. La documentación del código VHDL y del **SW** de PC se genera con **Doxygen**, a partir de los comentarios estructurados del propio código.

REQUISITOS Y CASOS DE USO

En este capítulo se recogen los requisitos capturados para el proyecto [VICON](#), así como los casos de uso derivados de ellos, que sirven de referencia para las fases de diseño y verificación descritas en los capítulos posteriores.

4.1 REQUISITOS DEL SISTEMA

La Tabla 4.1 resume el conjunto de requisitos capturados en conjunto con el cliente (el tutor en este caso) para nuestro proyecto. En la columna de prioridad, el valor “F” hace referencia a un requisito fundamental del proyecto y “D” a requisito deseable (requisito que se intentará cumplir). La Figura 4.1 muestra las relaciones entre requisitos de forma gráfica.

Tabla 4.1: Requisitos del proyecto.

Id	Nombre	Prioridad	Precedencia
R1.1	ConfigurarCamara	F	R8
R1.2	CapturarImagen	F	R1.1
R2.1	TransmitirVideo	F	R1.2
R2.2	SincronizarFrames	F	R2.1
R3.1	MostrarVideo	F	R2.1
R3.2	MostrarFPS	D	R3.1
R3.3	DisposicionGUI	D	R3.1
R3.4	DetectarCaras	F	R3.1
R4	RecibirComandos	D	R2.1
R4.1	EscribirRegistrosI2C	D	R4, R1.1
R4.2	ActivarCaptura	D	R4, R1.2
R5.1	ResolucionImagenes	D	R1.2
R5.2	ColorImagenes	D	R1.2
R5.3	TasaFotogramas	F	R2.1
R6	TiempoReal	D	R3.1
R7	ProtocoloFTDI	D	R2.1
R8	ConexionCamara	F	R9.2
R9.1	PlataformaProcesadora	F	

Id	Nombre	Prioridad	Precedencia
R9.2	PlataformaSensor	F	
R9.3	PlataformaHost	D	
R10.1	FechaSolicitudPrimeraOrd	F	
R10.2	FechaDefensaPrimeraOrd	F	R10.1
R10.3	FechaSolicitudSegundaOrd	F	
R10.4	FechaDefensaSegundaOrd	F	R10.3
R11.1	LenguajeProgramacionGUI	F	
R11.2	LenguajeProgramacionFPGA	F	

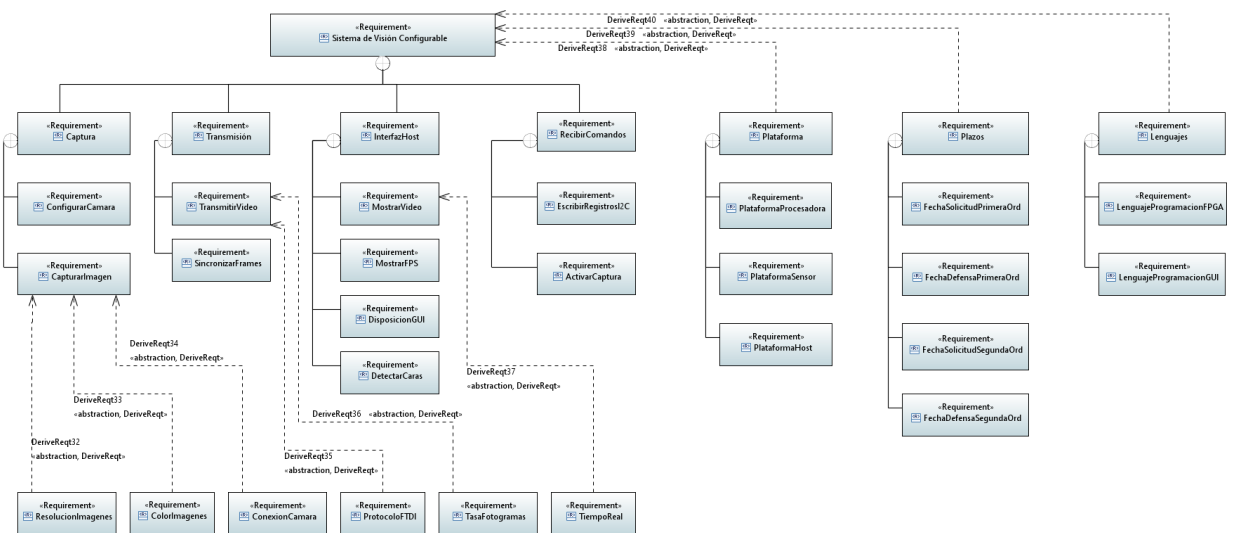


Figura 4.1: Diagrama general de requisitos.

4.1.1 Descripción de los requisitos funcionales

- R1.1 — **ConfigurarCamara**: El sistema configurará los registros de la cámara MT9V111 mediante el bus I2C durante el arranque.
- R1.2 — **CapturarImagen** : El sistema capturará los fotogramas procedentes de la cámara MT9V111.
- R2.1 — **TransmitirVideo** : El sistema transmitirá el vídeo capturado al Host (PC) a través del puerto USB mediante el módulo FT232H.
- R2.2 — **SincronizarFrames** : El sistema insertará un marcador de inicio de fotograma en el flujo de datos para que el Host (PC) delimite cada imagen.
- R3.1 — **MostrarVideo** : La interfaz gráfica del Host mostrará el vídeo recibido en una ventana en tiempo real.

- R3.2 — **MostrarFPS** : La GUI del Host mostrará los FPS actuales.
- R3.3 — **DisposicionGUI** : La GUI del Host estará formada por una ventana central donde se muestre el vídeo y botones laterales para las secciones de control.
- R3.4 — **DetectarCaras** : La aplicación del Host detectará la cara presente en el vídeo recibido mediante OpenCV.
- R4 — **RecibirComandos** : El sistema recibirá comandos enviados desde el Host (PC) a través del puerto USB.
- R4.1 — **EscribirRegistrosI2C** : El sistema escribirá en los registros de la cámara por I2C según el comando recibido del Host (PC).
- R4.2 — **ActivarCaptura** : El sistema activará o desactivará la captura de vídeo según el comando recibido del Host (PC).

4.1.2 Descripción de los requisitos no funcionales

- R5.1 — **ResolucionImágenes** : El vídeo transmitido tendrá una resolución mínima de 640×480 píxeles (VGA).
- R5.2 — **ColorImágenes** : El vídeo transmitido será en escala de grises.
- R5.3 — **TasaFotogramas** : El sistema transmitirá al Host (PC) un mínimo de 10 fotogramas por segundo bajo condiciones de luz suficientes.
- R6 — **TiempoReal** : El Host (PC) mostrará el vídeo de forma continua, sin acumular retardo creciente y sin almacenar la secuencia completa antes de mostrarla.
- R7 — **ProtocoloFTDI** : El protocolo de comunicación con el módulo FT232H será FIFO Asíncrono, siendo deseable el modo FIFO Síncrono.
- R8 — **ConexionCamara** : La cámara se insertará en los conectores Pmod JA y JXADC de la placa Basys 3.
- R9.1 — **PlataformaProcesadora** : El sistema de procesamiento se basa en la placa Basys 3 con FPGA Artix-7 XC7A35T.
- R9.2 — **PlataformaSensor** : El sensor de imágenes utilizado será el modelo MT9V111, conectado al sistema por medio de una PCB adaptadora de pines proporcionada por el cliente.
- R9.3 — **PlataformaHost** : El Host (PC) tendrá como sistema operativo Windows 11 y un mínimo de 8 GB de RAM.
- R10.1 — **FechaSolicitudPrimeraOrd** : El plazo de solicitud para la primera convocatoria Ordinaria es del 3/11/2025 al 30/06/2026.
- R10.2 — **FechaDefensaPrimeraOrd** : El plazo de defensa para la primera convocatoria Ordinaria acaba a los 20 días hábiles desde la solicitud y nunca después del 17/07/2026.

- R10.3 — **FechaSolicitudSegundaOrd** : El plazo de solicitud para la segunda convocatoria Ordinaria es del 27/08/2026 al 04/11/2026.
- R10.4 — **FechaDefensaSegundaOrd** : El plazo de defensa para la segunda convocatoria Ordinaria acaba a los 20 días hábiles desde la solicitud y nunca después del 30/11/2026.
- R11.1 — **LenguajeProgramacionGUI** : El lenguaje de programación utilizado para la captura de datos desde el Host y la creación de la GUI será preferiblemente C++.
- R11.2 — **LenguajeProgramacionFPGA** : El sistema se describirá mediante el lenguaje de descripción de hardware VHDL.

4.2 CASOS DE USO

Los actores externos con los que interactúa nuestro sistema se muestran en la Tabla 4.2. Las funcionalidades del sistema son, básicamente, capturar los fotogramas de la cámara, transmitirlos al ordenador para su visualización en tiempo real y su procesamiento, así como atender los comandos enviados desde el Host. El diagrama que muestra la relación del sistema (**VICON**) con los actores descritos en la Tabla 4.2 se muestra en la Figura 4.2.

Tabla 4.2: Actores del sistema.

Nombre	Descripción
Usuario	Persona que maneja la aplicación del Host: visualiza el vídeo, envía comandos y observa la detección de caras.
Cámara	Cámara MT9V111 externa al sistema, que proporciona los fotogramas de vídeo y es configurada por I2C.
Tiempo	Representa la temporización que desencadena de forma periódica la adquisición y transmisión de cada fotograma.

4.2.1 Descripción textual de los casos de uso

4.2.1.1 C1 — Capturar un fotograma de la cámara

CONTEXTO DE USO: A la frecuencia de captura, el sistema adquiere un fotograma de la cámara y extrae su información de luminancia para formar la imagen en escala de grises.

ACTOR PRINCIPAL: Tiempo.

PARTICIPANTES Y OBJETIVOS:

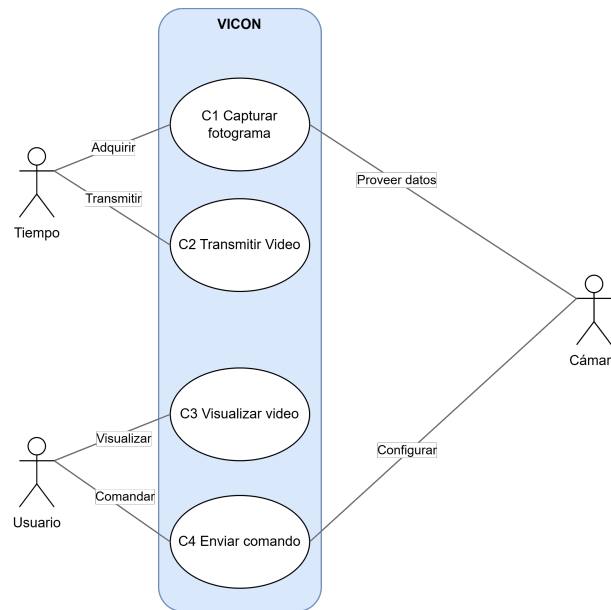


Figura 4.2: Diagrama de casos de uso.

Participante	Objetivo
Tiempo	Activar la adquisición de un nuevo fotograma.
Cámara	Proporcionar los datos del fotograma.

PRECONDICIONES: Cámara conectada y configurada por I2C.

GARANTÍAS MÍNIMAS: El sistema dispone de un fotograma en escala de grises listo para transmitir.

ESCENARIO DE ÉXITO PRINCIPAL:

- El Tiempo solicita al sistema la adquisición de un fotograma.
- El sistema recibe del sensor las líneas del fotograma.
- El sistema extrae la componente de luminancia de cada píxel.
- El sistema comienza a almacenar el fotograma en escala de grises para su transmisión.

ESCENARIO SECUNDARIO 1:

- La cámara no entrega datos válidos (sincronismo o iluminación insuficiente).
- El sistema descarta el fotograma y continúa con la siguiente adquisición.

4.2.1.2 C2 — Transmitir el vídeo al Host

CONTEXTO DE USO: A la frecuencia de captura, el sistema envía al ordenador el fotograma adquirido a través del enlace USB, delimitando cada imagen con un marcador de inicio.

ACTOR PRINCIPAL: Tiempo.

PARTICIPANTES Y OBJETIVOS:

Participante	Objetivo
Tiempo	Activar la transmisión del fotograma capturado.

PRECONDICIONES: Existe un fotograma capturado y el enlace USB (FT232H) está operativo.

GARANTÍAS MÍNIMAS: El fotograma queda disponible en el Host correctamente delimitado.

ESCENARIO DE ÉXITO PRINCIPAL:

- El Tiempo solicita al sistema la transmisión del fotograma.
- El sistema inserta el marcador de inicio de fotograma en el flujo de datos.
- El sistema envía los píxeles del fotograma al Host a través del FT232H.
- El Host recibe el fotograma y lo delimita con el marcador.

ESCENARIO SECUNDARIO 1:

- Se produce una sobreescritura del búfer por falta de lectura del Host a tiempo.
- El sistema descarta el fotograma para no acumular retardo.

4.2.1.3 C₃ — Visualizar el vídeo en el Host

CONTEXTO DE USO: El usuario observa en la aplicación del Host el vídeo recibido en tiempo real, junto con la tasa de fotogramas por segundo.

ACTOR PRINCIPAL: Usuario.

PARTICIPANTES Y OBJETIVOS:

Participante	Objetivo
Usuario	Visualizar el vídeo recibido.

PRECONDICIONES: La aplicación del Host está en ejecución y recibiendo fotogramas.

GARANTÍAS MÍNIMAS: El Host muestra el vídeo en una ventana y actualiza los FPS.

ESCENARIO DE ÉXITO PRINCIPAL:

- El Host recibe un fotograma completo delimitado por su marcador.
- El Host reconstruye la imagen en escala de grises.
- El Host muestra la imagen en la ventana central de la GUI.
- El Host actualiza el valor de FPS mostrado.

ESCENARIO SECUNDARIO 1:

- La aplicación detecta una cara en el fotograma mediante OpenCV.
- El Host resalta la cara detectada sobre el vídeo.

4.2.1.4 C4 — *Enviar un comando al sistema*

CONTEXTO DE USO: El usuario envía desde la aplicación del Host un comando al sistema (FPGA), bien para activar o desactivar la captura, bien para escribir un registro de la cámara por I2C.

ACTOR PRINCIPAL: Usuario.

PARTICIPANTES Y OBJETIVOS:

Participante	Objetivo
Usuario	Enviar un comando de control al sistema.
Cámara	Aplicar la configuración recibida por I2C.

PRECONDICIONES: El enlace USB está operativo y la cámara conectada.

GARANTÍAS MÍNIMAS: El sistema ejecuta el comando recibido.

ESCENARIO DE ÉXITO PRINCIPAL:

- El usuario selecciona un comando en la GUI del Host.
- El Host envía el comando al sistema por el enlace USB.
- El sistema decodifica el comando recibido.
- El sistema ejecuta la acción solicitada (activar/desactivar captura, encender/apagar LED, alternar entre imagen real y sintética o escribir un registro por I2C).

ESCENARIO SECUNDARIO 1:

- El comando recibido no es válido.
- El sistema ignora el comando y mantiene su estado actual.

DESCRIPCIÓN Y ARQUITECTURA DEL SISTEMA

En este capítulo se describe la arquitectura del sistema [VICON](#), presentando en primer lugar una visión global de sus bloques principales, para después detallar la arquitectura física, el hardware empleado y, por último, la arquitectura lógica interna de la [FPGA](#).

5.1 ARQUITECTURA GLOBAL

VICON es un sistema de adquisición y transmisión de imagen en tiempo real cuya arquitectura se divide en tres bloques principales: la placa de desarrollo Basys 3, equipada con una [FPGA](#) Xilinx Artix-7 sobre la que se implementa toda la lógica de procesamiento, el sensor de imagen MT9V111 y el PC de visualización. La Figura 5.1 muestra el diagrama de bloques del sistema completo en alto nivel. El flujo de datos de imagen sigue el camino en sentido de derecha a izquierda: sensor MT9V111 → FPGA → PC. El flujo de comandos sigue el camino inverso: PC → FPGA → sensor MT9V111.

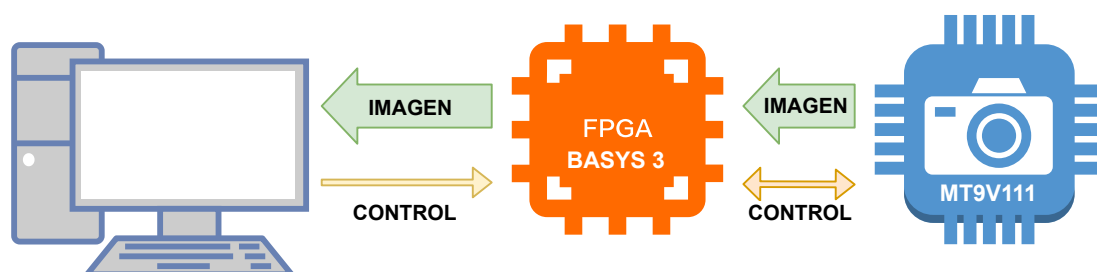


Figura 5.1: Arquitectura global del sistema VICON.

Al descomponer la arquitectura global en sus componentes de nivel inmediatamente inferior, se distinguen los siguientes bloques (representados en la Figura 5.2):

- Sensor MT9V111: expone, tal y como se indica en su hoja de datos [2], por un lado, los **datos** de la imagen capturada y, por otro, los **registros de control**, organizados en dos páginas y accesibles mediante [I2C](#) (Inter-Integrated Circuit).
- FPGA Basys 3: en la imagen se muestran los módulos que hacen de interfaz con el exterior: la adquisición de imágenes, el control del sensor por [I2C](#) y la comunicación con el PC a través del chip FT232H. Uno de los retos principales del diseño implica diseñar tanto estos módulos como la comunicación entre ellos, gestionando de manera correcta los cruces de reloj presentes. Además, la FPGA

implementa una **FSM** (Finite State Machine) que gestiona procesos tales como la configuración inicial del sensor o la gestión de comandos recibidos por el PC.

- PC: encargado de realizar el procesamiento de imágenes y de mostrar la interfaz gráfica de usuario. Esta presenta las imágenes, la información asociada (p. ej., los **FPS** (Frames por segundo)) y los controles disponibles del sistema.
- Chip FT232H: gestiona la comunicación entre la FPGA y el PC, proporcionando un enlace de alta velocidad. Convierte el bus paralelo de 8 bits que expone la FPGA en modo *Synchronous FIFO* en un flujo serie sobre USB 2.0 *High-Speed*.

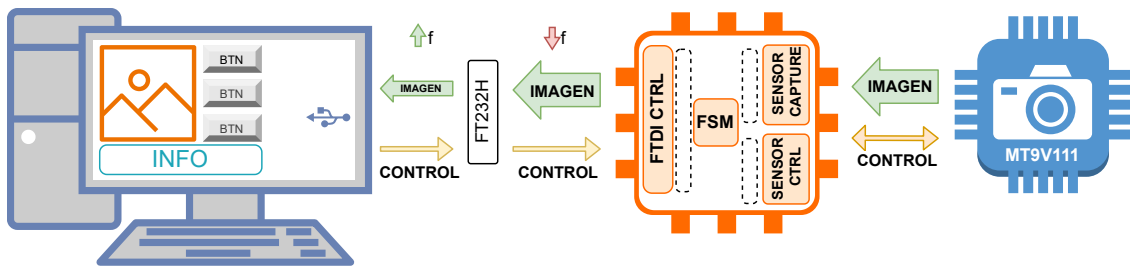


Figura 5.2: Descomposición parcial de la arquitectura global.

5.2 ARQUITECTURA FÍSICA

La arquitectura física describe los elementos Hardware del sistema y las interfaces que los conectan. La Figura 5.3 recoge el diagrama de bloques físico de **VICON**. El sistema se articula en torno a la placa Basys 3, que aloja la FPGA Artix-7 donde reside toda la lógica; el sensor MT9V111, montado sobre una **PCB** (Printed Circuit Board) adaptadora que reordena sus pines hacia los conectores Pmod; y el módulo FT232H, que enlaza la FPGA con el PC anfitrión a través de USB 2.0. El sensor se conecta a los Pmod JA y JXADC, mientras que el FT232H ocupa otros dos Pmod. El detalle de cada componente se recoge en la sección 5.3.

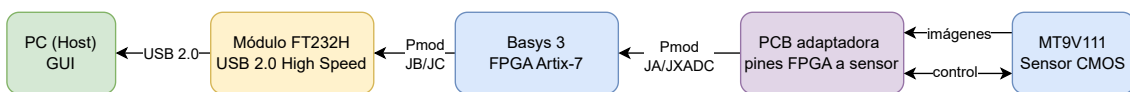


Figura 5.3: Arquitectura física del sistema VICON (diagrama de bloques).

5.3 HARDWARE

5.3.1 FPGA Basys 3

La Basys 3 es una plataforma de desarrollo de Digilent basada en el dispositivo Xilinx XC7A35T-1CPG236C de la familia Artix-7 [7]. Dispone de:

- 33.280 celdas lógicas

- 1.800 Kb de BRAM
- oscilador de 100 MHz en placa
- 16 *switches*
- 5 pulsadores
- 16 LEDs
- 4 displays de 7 segmentos
- 4 conectores PMOD. Para este proyecto se utilizan los conectores PMOD para conectar el sensor MT9V111 y el chip FT232H.

5.3.2 Sensor CMOS MT9V111

El sensor MT9V111 de ON Semiconductor es un sensor de imagen CMOS de resolución VGA (640×480 píxeles). Sus características principales son:

- Salida de datos en varios formatos. El que se ha utilizado en este proyecto ha sido el formato YCbCr 4:2:2 con 8 bits por ciclo de píxel. El formato de salida es configurable a través de los registros internos del sensor, accesibles mediante la interfaz [I2C](#); en el presente proyecto se confirmó en la fase inicial que el protocolo configurado era el correcto.
- Reloj de píxel (PIXCLK) generado por el propio sensor a partir del reloj maestro (MCLK) suministrado por la FPGA.
- Señales de sincronismo de trama (FVALID) y de línea (LVALID).
- Interfaz de configuración [I2C](#) con la dirección configurada a 0x5C (7 bits).
- Registros organizados en dos páginas: página 0 (sensor core) y página 1 ([IFP](#) (Image Flow Processor)).

Durante la fase inicial de diseño, antes de escribir la lógica [RTL](#) de `frame_capture` (Sección [6.2.3](#)), se capturaron las señales reales del sensor con un analizador lógico externo (utilizando como visualizador el Software Logic 2) para confirmar empíricamente el comportamiento descrito en el *datasheet* [\[2\]](#). La Figura [5.4](#) muestra dicha captura, con la nomenclatura del sensor (VSYNC/HREF, equivalentes a FVALID/LVALID en el resto de la memoria): el reloj de píxel PIXCLK conmutando de forma continua, el pulso de VSYNC delimitando el inicio de fotograma, la señal HREF delimitando cada línea activa, y el bus de datos D[7:0] conmutando únicamente durante las ventanas en las que HREF está a nivel alto. Esta caracterización previa permitió validar la interpretación del protocolo frente al *datasheet* antes de desarrollar un diseño RTL concreto, y sirvió de base para la máquina de estados de `frame_capture`.

En la configuración actual, el sensor opera a 30 [FPS](#) con resolución [VGA](#) entregando imágenes en escala de grises (componente Y del espacio YCbCr), dado que los valores de Cb y Cr se descartan con el fin de aumentar la tasa de frames por segundo.

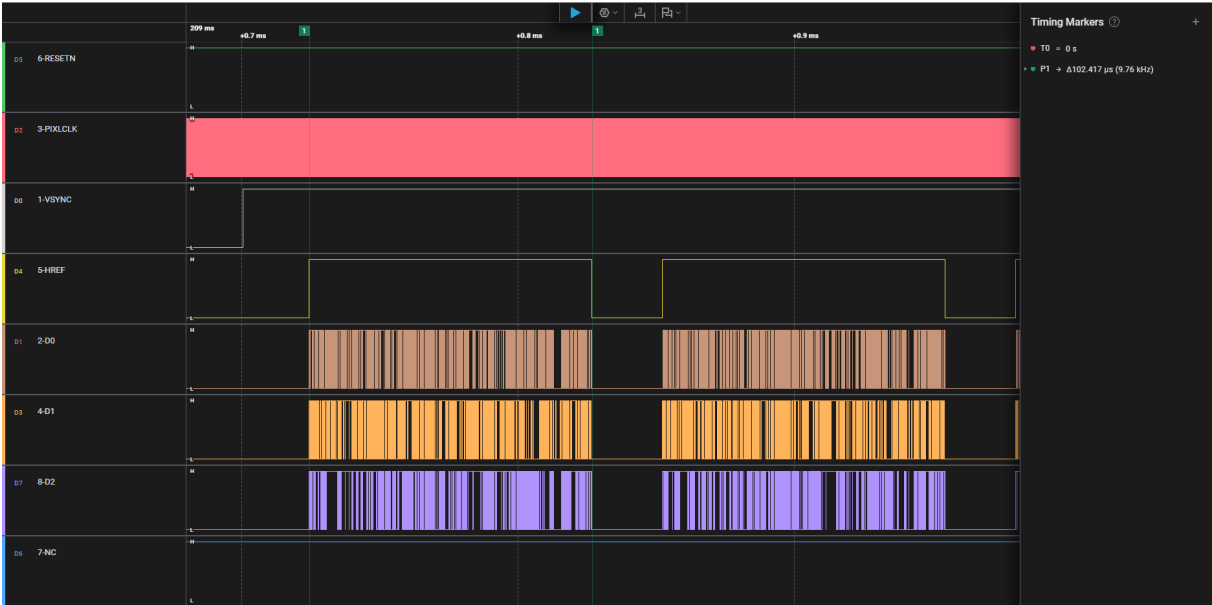


Figura 5.4: Captura con analizador lógico de las señales del MT9V111 (RESETN, PIXLCLK, VSYNC, HREF, D[7:0]) durante la fase inicial de caracterización.

La Tabla 5.1 recoge la asignación de pines de la FPGA para el sensor MT9V111, reflejada en el fichero de restricciones Basys3_GPIO.xdc.

Tabla 5.1: Asignación de pines de la FPGA para el sensor MT9V111.

Señal (VHDL)	Pin FPGA	Función
mt_clk_o	L2	MCLK: reloj maestro hacia el sensor (27 MHz, generado por el MMCM (Mixed-Mode Clock Manager); valor máximo admitido por el sensor según su hoja de características).
mt_pixclk_i	H1	PIXCLK: reloj de píxel generado por el propio sensor a partir de MCLK (27 MHz; <code>create_clock -period 37.037</code>). En la configuración por defecto del sensor, $f_{PIXCLK} = f_{MCLK}$; si se requiriera una relación distinta sería necesario configurar los registros del PLL (Phase-locked loop) interno del sensor a través de I2C . Pin no <i>clock-capable</i> (<code>CLOCK_DEDICATED_ROUTE FALSE</code>).
mt_reset_n_o	J3	RESET#: reset del sensor, activo a nivel bajo.
mt_fvalid_i	H2	FVALID (<i>frame valid</i>): validez de fotograma.
mt_lvalid_i	J2	LVALID (<i>line valid</i>): validez de línea.
mt_data_i[7:0]	D0UT[7:0]: bus de datos de imagen (YCbCr 4:2:2 / componente Y). Pines, de bit 0 a bit 7: L3, M3, M2, M1, N2, N1, J1, K2.	
i2c_sclk_io	G3	SCL (Serial Clock): reloj I2C (<i>pull-up</i> interno de la FPGA activado).
i2c_sdata_io	G2	SDA (Serial Data): datos I2C (<i>pull-up</i> interno de la FPGA activado).

La conexión del sensor a la BASYS se realiza tal y como se muestra en la siguiente imagen.

5.3.3 Chip FT232H

El FT232H de FTDI es un convertidor USB 2.0 a interfaz serie/paralela de alta velocidad. En este proyecto se configura en modo *Synchronous FIFO*, seleccionado mediante la herramienta FT Prog de FTDI, que permite programar la configuración de la interfaz en la [EEPROM](#) (Electrically Erasable Programmable Read-Only Memory) externa del propio chip. Este modo proporciona:

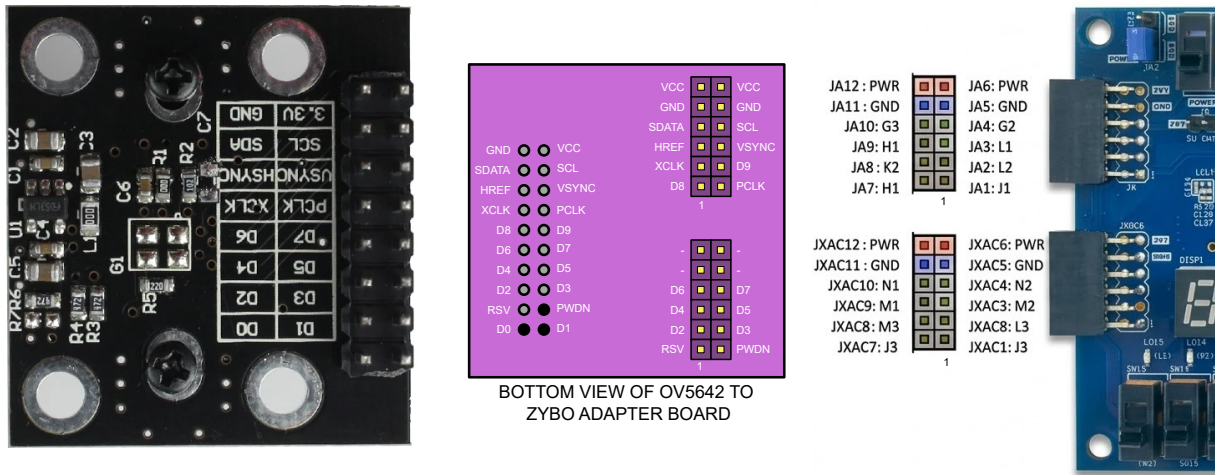


Figura 5.5: Conexión del sensor CMOS a la Basys 3.

- Un bus de datos bidireccional de 8 bits (ADBUS[7:0]).
- Un bus de control (ACBUS) que incluye las señales RXF#, TXE#, RD#, WR#, OE# y CLKOUT.
- Un reloj **síncrono** de 60 MHz generado internamente (CLKOUT).
- Tasa de transferencia teórica de hasta 40 MB/s en modo de FIFO síncrona [1].

El módulo UM232H-B se conecta a la Basys 3 mediante dos conectores *Pmod*: uno para ADBUS[7:0] y otro para ACBUS[7:0]. La Tabla 5.2 recoge el bus de datos y la Tabla 5.3 el bus de control, incluyendo el color del cable de cada línea del *cable Pmod* tal y como se cableó físicamente (información reflejada en los comentarios del fichero Basys3_GPI0.xdc).

Tabla 5.2: Asignación de pines de la FPGA para el bus de datos del FT232H (ADBUS).

Señal (VHDL)	Pin FPGA	Cable Pmod	Línea FT232H
ftdi_adbus_io[0]	A14	Marrón (pin 1)	Do
ftdi_adbus_io[1]	A15	Rojo (pin 7)	D1
ftdi_adbus_io[2]	A16	Naranja (pin 2)	D2
ftdi_adbus_io[3]	A17	Amarillo (pin 8)	D3
ftdi_adbus_io[4]	B15	Verde (pin 3)	D4
ftdi_adbus_io[5]	C15	Azul (pin 9)	D5
ftdi_adbus_io[6]	B16	Morado (pin 4)	D6
ftdi_adbus_io[7]	C16	Gris (pin 10)	D7

Tabla 5.3: Asignación de pines de la FPGA para el bus de control del FT232H (ACBUS).

Señal (VHDL)	Pin FPGA	Cable Pmod	Línea FT232H
ftdi_acbus_io[0]	K17	Rojo (pin 1)	RXF# (entrada): FIFO de TX no vacía.
ftdi_acbus_io[1]	L17	Naranja (pin 7)	TXE# (entrada): FIFO de RX no llena.
ftdi_acbus_io[2]	M18	Amarillo (pin 2)	RD# (salida): activa lectura de un byte.
ftdi_acbus_io[3]	M19	Verde (pin 8)	WR# (salida): activa escritura de un byte.
ftdi_acbus_io[4]	N17	Azul (pin 3)	SIWU# (salida, fija a '1').
ftdi_acbus_io[5]	P17	Morado (pin 9)	CLKOUT (entrada): reloj síncrono de 60 MHz. Pin no <i>clock-capable</i> (origen de la violación de <i>timing</i> de la Sección 8.1.1).
ftdi_acbus_io[6]	P18	Gris (pin 4)	OE# (salida): habilita al FT232H a conducir ADBUS.
ftdi_acbus_io[7]	R18	Blanco (pin 10)	PWRSV# (salida, fija a '1').

5.4 ARQUITECTURA LÓGICA

La arquitectura lógica define los componentes descritos por lenguaje, *SW* o *HW* (Hardware), del sistema y las relaciones que existen entre ellos con independencia de su realización física. En *VICON* estos componentes se reparten entre dos entornos muy distintos: la lógica descrita en *VHDL* que se sintetiza sobre la FPGA y la aplicación *SW* que se ejecuta en el PC (Host). La Figura 5.6 presenta una visión general de ambos componentes y de las interfaces que los comunican, mientras que la Figura 5.7 detalla la estructura interna de la lógica de la FPGA a nivel de módulo y señal.

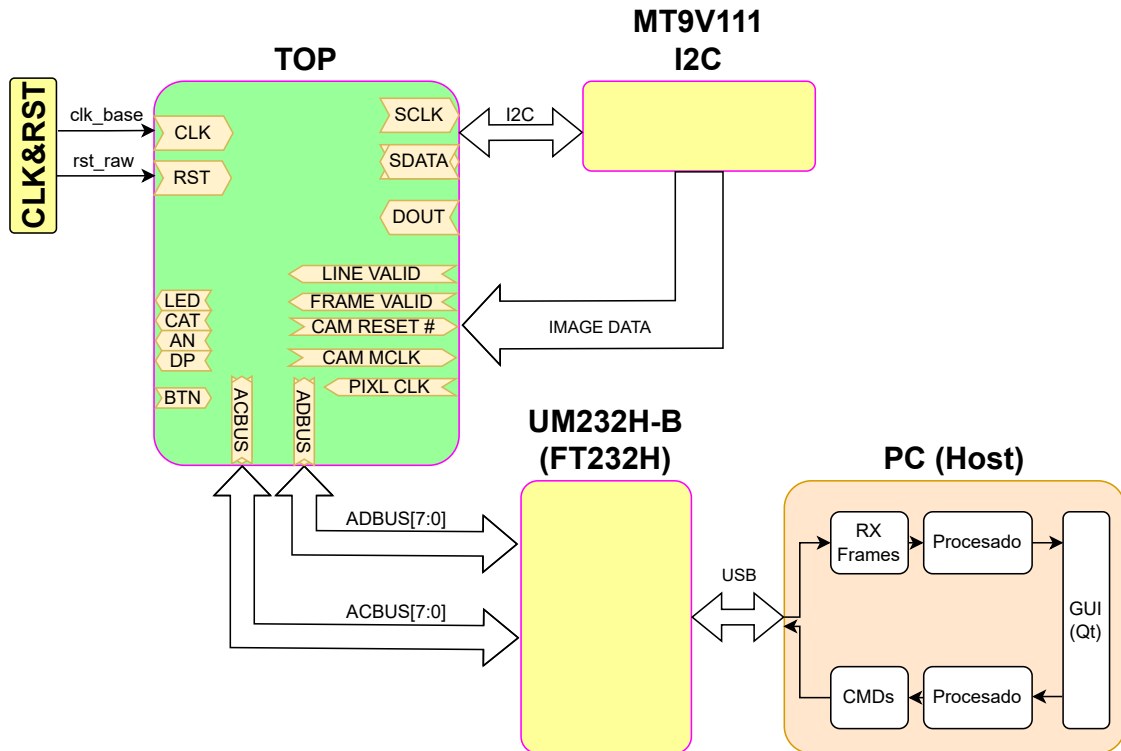


Figura 5.6: Arquitectura lógica del sistema VICON: visión general.

5.4.1 Componentes de la FPGA

Todos los componentes de la lógica programable se integran bajo el módulo TOP, que actúa como contenedor: instancia cada submódulo, resuelve sus interconexiones y expone hacia el exterior los puertos de la placa. Ninguno de los submódulos tiene existencia propia fuera del TOP, dado que su ciclo de vida queda ligado al del diseño sintetizado.

Los componentes identificados son los siguientes:

- **i2c_controller**: implementa el maestro **I2C** que configura los registros del sensor MT9V111 y permite su lectura posterior. Expone hacia el exterior las señales **SCL** y **SDA**.
- **frame_capture**: recibe el flujo de píxeles del sensor, extrae la componente de luminancia, inserta los marcadores de inicio de trama y regula la tasa de fotogramas entregada.
- **ftdi_controller**: gestiona el diálogo con el chip FT232H en modo *Synchronous FIFO*, tanto en el envío de imagen como en la recepción de comandos.
- **cmd_processor**: decodifica los comandos recibidos y ejecuta la acción asociada a cada uno.

- `mt9v111_image`: genera un patrón de imagen sintético que emula el protocolo de salida del sensor, permitiendo validar la cadena de transmisión independientemente del Hardware.
- **FSM principal**: coordina la secuencia de configuración del sensor y habilita la captura una vez completada. Además, es la encargada de discernir si le ha llegado un comando y realizar la acción correspondiente. Actualmente **se prioriza la recepción de un comando frente al envío de trama**, ya que se entiende que es una situación más puntual e importante que la transmisión de imagen.

A estos componentes propios se suman dos **IPs** del catálogo del fabricante: el **MMCM** (`clk_wiz_0`), que genera el reloj del sistema, y la **FIFO** asíncrona (`fifo_generator_0`), que desacopla la captura de la transmisión.

5.4.2 Relaciones entre componentes y dominios de reloj

La Figura 5.7 muestra la interconexión de estos módulos, utilizando nombres representativos que pueden no coincidir con el nombre exacto de la instancia. Los componentes no comparten un único reloj, sino que se reparten entre tres dominios independientes (cuatro si se contabiliza la comunicación I2C).

El módulo `frame_capture` opera en el dominio del reloj de píxel que impone el sensor, mientras que `ftdi_controller` lo hace en el dominio del reloj que suministra el propio FT232H. La relación entre ambos no puede ser directa: se comunican a través de la **FIFO** asíncrona, cuyo puerto de escritura pertenece al primer dominio y cuyo puerto de lectura pertenece al segundo. La FIFO actúa así como **elemento de desacoplo** entre la producción y el consumo de datos de imagen.

El resto de relaciones entre dominios corresponde a señales de control y se resuelve mediante sincronizadores de doble flip-flop, representados en la figura entre los módulos implicados: las señales de habilitación de captura y de selección de fuente de imagen cruzan desde el dominio de control hacia el dominio de píxel, mientras que los comandos decodificados cruzan en sentido inverso, desde el dominio del FT232H hacia el de control. El detalle de estos cruces, así como las restricciones de temporizado asociadas, se detalla en la Sección 6.3.

La selección entre imagen real e imagen sintética se resuelve mediante un multiplexor controlado por la señal de selección sincronizada, de modo que `mt9v111_image` y el sensor físico son fuentes intercambiables para `frame_capture` por medio de un comando de usuario.

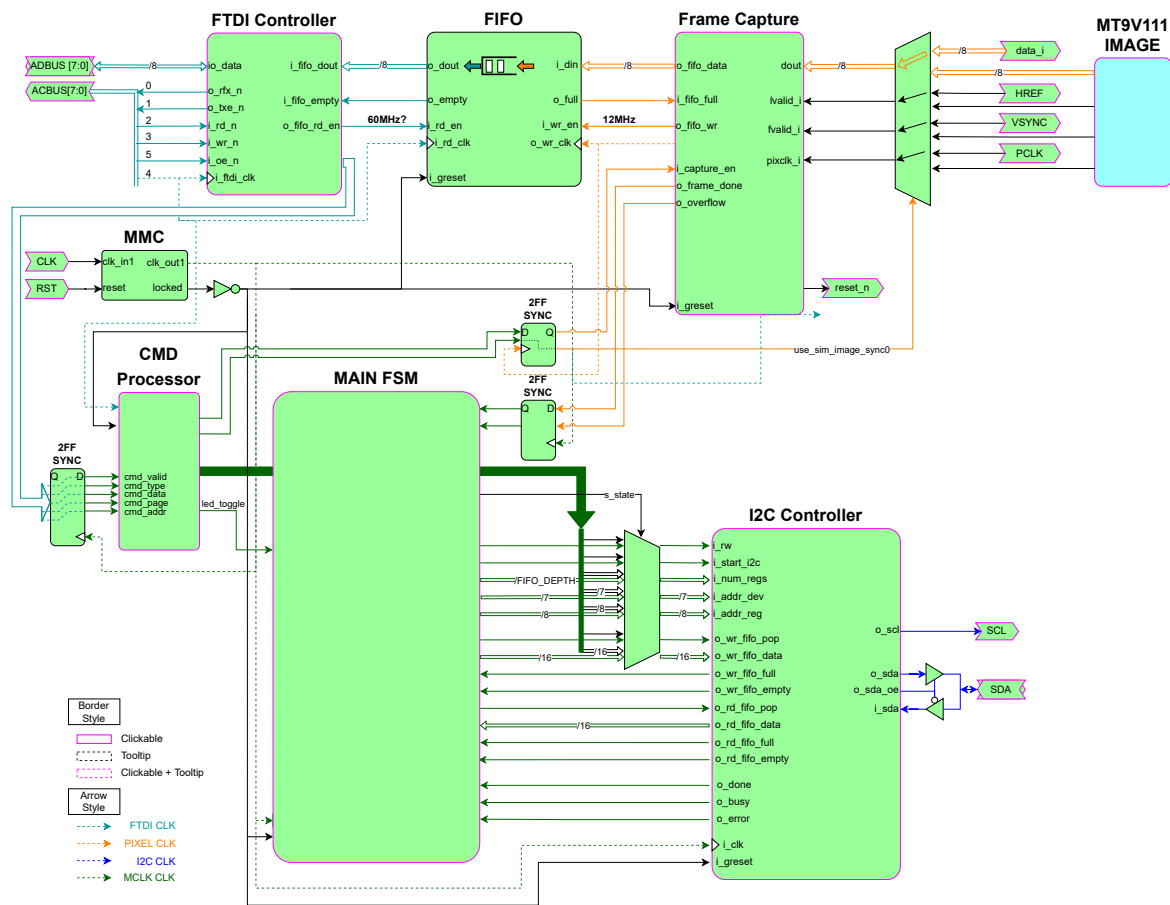


Figura 5.7: Arquitectura lógica de la FPGA: módulos, señales y dominios de reloj.

5.4.3 Componentes del Host

En el lado del PC, la aplicación `vicon_view` agrupa los componentes software encargados de recibir, procesar y presentar el vídeo, así como de enviar los comandos de control. Como se aprecia en la Figura 5.6, se identifican cuatro componentes:

- **RX Frames:** recibe el flujo de bytes procedente del enlace USB, localiza los marcadores de inicio de trama y reconstruye cada fotograma completo.
- **Procesado:** interpreta los fotogramas recibidos como imágenes en escala de grises y aplica sobre ellos el procesamiento de visión.
- **CMDs:** construye los comandos de control y los envía hacia la FPGA a través del mismo enlace.
- **GUI (Qt):** presenta el vídeo, la información asociada y los controles con los que el usuario interactúa con el sistema.

La relación entre estos componentes describe un lazo cerrado: el flujo de imagen asciende desde la recepción hasta la interfaz gráfica, mientras que las acciones del usuario sobre la interfaz descienden en forma de comandos hacia la FPGA. Ambos sentidos comparten un único enlace físico (USB), lo que condiciona el diseño de la aplicación y se detalla en el Capítulo 7.

DESARROLLO

Este capítulo describe el desarrollo del sistema [VICON](#), comenzando por la organización del repositorio y del proyecto de Vivado, para después detallar el diseño [RTL](#) de cada uno de los módulos que lo componen, la estrategia seguida para la gestión de los cruces de dominio de reloj ([CDC](#)) presentes en el diseño y, por último, el proceso de verificación mediante simulación.

6.1 ESTRUCTURA DEL PROYECTO

El proyecto se organiza según la siguiente estructura de directorios:

- 01-Sources/: fuentes [RTL](#) del diseño (TOP.vhd, módulos, paquetes de constantes).
- 02-IPs/: [IPs](#) generadas en Vivado fuera del directorio del proyecto.
- 03-BD/: directorio reservado para diagramas de bloques. En este proyecto se ha optado por realizarlo íntegramente con [VHDL](#), pero se mantiene el directorio como parte del estándar definido..
- 04-Constraints/: Archivo de restricciones de pines y de tiempo relativos a la placa Basys 3.
- 05-TestBench/: fuentes del banco de pruebas (agentes, generadores de reloj, testbench principal).
- 06-Project/: proyecto Vivado con las [IPs](#) generadas y los ficheros de constraints (.xdc).
- 07-Versions/: versiones estables (releases) que incluyen el bitstream y el ejecutable de la [GUI](#) (Graphical User Interface). A fecha de entrega de este documento la última versión estable es la 3.0.
- 08-Repositories/: directorio reservado para librerías, interfaces o archivos de inicialización de memorias. En este proyecto no se hace uso de ellos, aunque de nuevo se mantiene el directorio como parte del estándar definido.
- 09-Doxygen/: Archivos de documentación automática con Doxygen.
- 10-FrontEnd/: Archivos de [SW](#) relativo a la [GUI](#).

Los paquetes de constantes `config_pkg.vhd` y `top_pkg.vhd` centralizan todos los parámetros configurables del diseño (resolución, frecuencias, protocolo de sustitución de bytes reservados, etc.), favoreciendo la reutilización de los bloques y su configuración modificando únicamente estos archivos.

6.2 RTL DESIGN

6.2.1 TOP

El módulo `TOP.vhd` instancia y conecta todos los submódulos del diseño. Sus responsabilidades principales son:

- Instanciar el `MMCM` (`clk_wiz_0`) para generar `s_mclk` (100 MHz) y `mt_clk_o` (30 MHz hacia el sensor).
- Gestionar la señal de reset global (`s_rst_final`), activa hasta que el `MMCM` bloquea (`locked='1'`).
- Implementar la `FSM` de configuración del sensor (`p_fsm`), que coordina la inicialización por medio de `I2C` del sensor MT9V111 antes de habilitar la captura.
- Implementar los sincronizadores 2FF para los cruces de dominio `s_mclk`→`mt_pixclk_i` de las señales `s_use_sim_image` y `s_cap_en`.
- Instanciar la FIFO asíncrona (`fifo_generator_0`) que desacopla el dominio de escritura (`mt_pixclk_i`) del dominio de lectura (`ftdi_clk`).

La `FSM` de configuración (`p_fsm`) sigue la secuencia de estados mostrada en la Figura B.1 (Anexo B, dado su tamaño). A muy alto nivel, la secuencia recorre cuatro bloques:

1. **Reset del sensor** (`ST_CAM_RESET_ASSERT` / `ST_CAM_RESET_WAIT`): mantiene `mt_reset_n_o` a nivel bajo durante `g_MT9V111_RESET_HOLD_US` y espera después `g_MT9V111_RESET_WAIT_US` para la estabilización del PLL del sensor.
2. **Selección de página y verificación de identidad** (`ST_PAGE_SEL_*`, `ST_CHIPID_RD_*`): antes de escribir ningún registro, la FSM selecciona la página o (*Core*) y lee por `I2C` el registro de *Chip ID* (`0xFF`), comparándolo contra el valor esperado del MT9V111 (`0x823A`). Si la lectura no coincide, la FSM transita a `ST_ERROR` sin llegar a configurar el sensor, evitando escribir registros sobre un dispositivo no identificado correctamente (p. ej. por un fallo de conexión de la PCB adaptadora).
3. **Escritura y verificación de la tabla de configuración** (`ST_CFG_PAGE_*`, `ST_CFG_WR_*`, `ST_CFG_RD_*`, `ST_CFG_NEXT`): para cada entrada de `c_CFG_TABLE` se selecciona la página correspondiente, se escribe el registro y a continuación **se relee para verificarlo** (*write+verify*); si la relectura no coincide con el valor escrito, la FSM va a `ST_ERROR`.

4. **Estados finales** (ST_FINISH / ST_ERROR): en ST_FINISH el sistema queda habilitado para capturar vídeo y para atender comandos del PC (véase la Sección 6.2.5); en ST_ERROR el sistema queda bloqueado. Ambos estados se señalizan de forma permanente en los LEDs 0 y 1 de la Basys 3 respectivamente (LED 0 encendido = configuración correcta, LED 1 encendido = fallo de configuración), lo que permite diagnosticar un fallo de arranque del sensor sin necesidad de abrir el ILA.

Como parte de esa inicialización, la constante `c_CFG_TABLE` (definida en `T0P.vhd`) recoge la tabla de 13 registros que se escriben por `I2C` en el sensor durante el arranque, indicando para cada uno la página (Core o `IFP`), la dirección y el valor. Entre ellos se incluyen los registros de *blanking* horizontal y vertical (R5, R6) y el ancho de obturador (R33), que junto con varios registros de la página `IFP` determinan la tasa de fotogramas efectiva que entrega el sensor.

El registro R55 (página `IFP`, dirección `0x37`) fija de forma directa la tasa de fotogramas mínima del sensor y, según el *datasheet*, es necesario ajustarlo para que el MT9V111 alcance de forma robusta la tasa objetivo de 30 FPS (requisito R5.3): sin él, la combinación por defecto de los demás registros de `c_CFG_TABLE` no garantiza los 30 fps requeridos. En la versión actual del diseño este registro **no se incluye en la tabla de arranque automático**; en su lugar, se envía manualmente como comando `I2C` desde la interfaz de `vicon_view` una vez arrancado el sistema (trama 1 `0x37 0x0080`), tal y como se describe en el Apéndice A (Figura A.2). Esto se hace así para verificar el cambio de la tasa de envío de imágenes.

6.2.2 I2C Controller

El módulo `i2c_controller.vhd` implementa un maestro `I2C` parametrizable que soporta escritura y lectura de registros de 16 bits. Opera en el dominio `s_mclk` (100 MHz) y genera la frecuencia de `SCL` mediante un divisor de reloj configurable a través del genérico `g_I2C_FREQ_HZ`, fijado en 200 kHz. Aunque el estándar `I2C` permite hasta 400 kHz en modo *Fast-mode*, esta frecuencia resultó poco fiable: las resistencias de *pull-up* internas de la `FPGA` en los pines de `SCL` y `SDA` son demasiado débiles para garantizar los tiempos de subida del bus a 400 kHz, lo que **provocaba errores esporádicos**. Reducir la frecuencia a 200 kHz eliminó este problema sin penalizar de forma apreciable el tiempo de configuración del sensor, que se realiza una única vez durante el arranque.

La dirección del esclavo (`addr_dev_i`) se recibe como parámetro de entrada y no está fijada en el propio controlador, de modo que el mismo módulo podría reutilizarse con otros dispositivos `I2C`; en el caso de VICON, el sensor MT9V111 responde a la dirección de 7 bits `0x5C`.

PROTOCOLO I2C MT9V111 El sensor MT9V111 sigue el protocolo estándar de escritura y lectura de registros de 16 bits en `I2C`. La escritura sigue la secuencia habitual de una única fase:

1. START + dirección esclavo (W) + ACK.
2. Dirección del registro (8 bits) + ACK.

3. Byte alto del dato (*MSB*) + ACK.
4. Byte bajo del dato (*LSB*) + ACK + STOP.

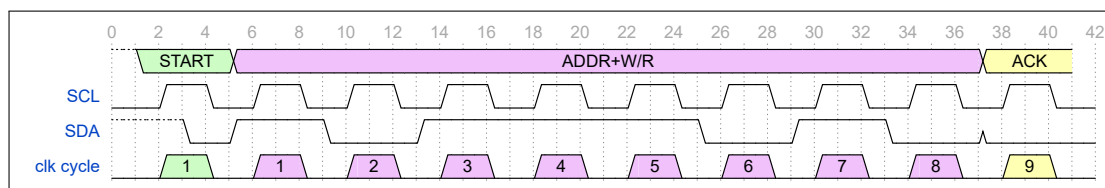
La lectura, en cambio, requiere una condición de *Repeated START* (sin STOP intermedio) para cambiar de dirección de transferencia sin liberar el bus:

1. Fase de escritura de la dirección del registro: START + dirección esclavo (W) + ACK + dirección registro + ACK.
2. *Repeated START* + dirección esclavo (R) + ACK.
3. Fase de lectura del dato: MSB dato + ACK + LSB dato + NACK + STOP.

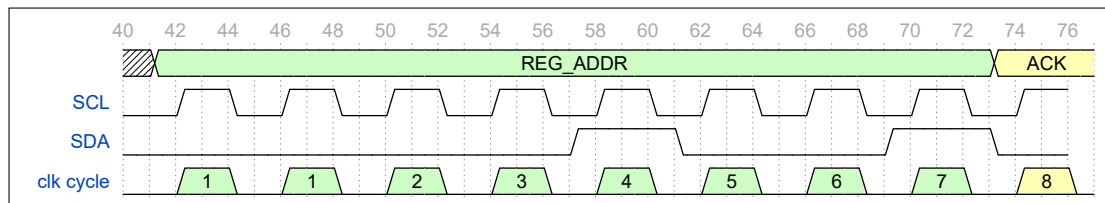
Esta secuencia es la que implementa `i2c_controller.vhd`: tras recibir el ACK de la dirección de registro (estado `ST_DECIDE_AFTER_REG_ADDR`), si la transacción es de lectura el controlador no pasa por `ST_STOP_0` sino que activa `s_start_rd_mode` y vuelve directamente a `ST_START_0`, generando el *Repeated START*. El comportamiento se verificó comparando la forma de onda del bus I2C capturada en simulación y en hardware con el datasheet del sensor (Figura 6.1) [2], y el agente de simulación `mt9v111_i2c.vhd` se implementó para reproducir exactamente este comportamiento.

El bit de *START* se genera cuando `SDA` baja mientras `SCL` está en alto; el ciclo completo de cada transacción sigue las 4 fases de `SCL` mostradas en la Figura 6.2.

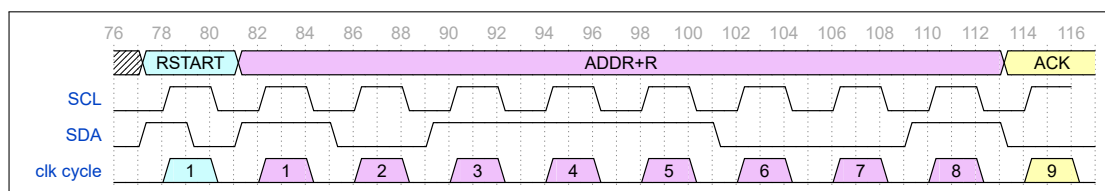
MÁQUINA DE ESTADOS La Tabla 6.1 resume los 37 estados de la máquina (véase también el diagrama de estados completo en la Figura B.2, Anexo B), agrupados por fase del protocolo. Cada byte transmitido (dirección, dirección de registro o dato) se descompone en 4 subestados que recorren las 4 fases de `SCL` de la Figura 6.2; por claridad, la tabla los agrupa en una sola fila. Nótese que el número de estados es relativamente bajo para un protocolo con tantas fases porque la FSM reutiliza los mismos estados `ST_TX_*/ST_RACK_*` y `ST_RX_*/ST_MACK_*` para *todas* las transferencias de byte (dirección de esclavo, dirección de registro y datos), retornando en cada caso al punto de decisión correspondiente en vez de duplicar estados por cada tipo de byte.



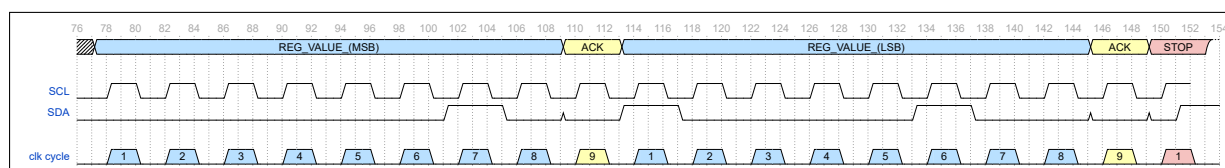
(a) Envío de dirección de esclavo



(b) Envío de dirección del registro + W



(c) Repeated START y envío de dirección del registro + R



(d) Envío de valor del registro: MSB & LSB

Figura 6.1: Tramas para escritura y lectura de 16 bits en el bus I2C del sensor MT9V111.

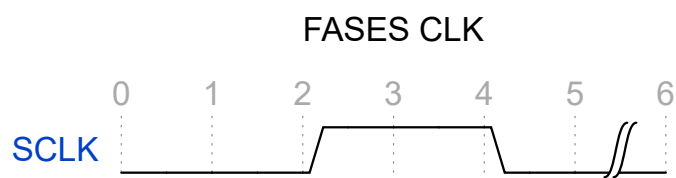


Figura 6.2: Cronograma de las 4 fases de SCLK.

Tabla 6.1: Estados de la FSM de `i2c_controller.vhd`.

Estado(s)	Función
ST_IDLE	Espera a que se solicite una transacción (<code>start_i2c_i</code>).
ST_START_0..3	Genera la condición de START (<i>SDA</i> baja con <i>SCL</i> alta).
ST_TX_0..3	Transmite un byte (dirección de esclavo, de registro o dato).
ST_RACK_0..3	Libera <i>SDA</i> y muestrea el ACK del esclavo.
ST_DECIDE_AFTER_ADDR_WR	Decide el siguiente byte tras el ACK tras la dirección de escritura.
ST_DECIDE_AFTER_REG_ADDR	Decide entre continuar en escritura o iniciar el <i>Repeated START</i> de lectura.
ST_LOAD_DATA_H	Carga el byte alto (<i>MSB</i>) del dato a escribir.
ST_DECIDE_AFTER_DATA_H	Decide tras el ACK del <i>MSB</i> .
ST_DECIDE_AFTER_DATA_L	Decide tras el ACK del <i>LSB</i> : más registros o STOP.
ST_LOAD_NEXT_DATA_H	Carga el <i>MSB</i> del siguiente registro (escritura de ráfaga).
ST_DECIDE_AFTER_ADDR_RD	Decide tras el ACK de la dirección de esclavo en modo lectura.
ST_RX_0..3	Recibe un byte del esclavo (<i>MSB</i> o <i>LSB</i>).
ST_DECIDE_AFTER_RX_H	Decide tras recibir el <i>MSB</i> : continúa a <i>LSB</i> .
ST_DECIDE_AFTER_RX_L	Decide tras recibir el <i>LSB</i> : envía NACK final.
ST_MACK_0..3	Genera el ACK/NACK del maestro tras una lectura.
ST_STOP_0..3	Genera la condición de STOP.
ST_DONE	Señaliza la finalización correcta de la transacción (<code>o_done</code>).
ST_ERROR_STOP / ST_ERROR	Generan STOP tras un fallo (p. ej. ausencia de ACK) y señalizan el error (<code>o_error</code>).

6.2.3 Frame Capture

El módulo `frame_capture.vhd` captura los datos de imagen procedentes del sensor MT9V111 y los escribe en la FIFO asíncrona. Opera en el dominio `mt_pixclk_i` (25 MHz). Sus funciones principales son:

- Detectar el inicio y fin de trama mediante las señales `fvalid_i` y `lvalid_i`.
- Extraer únicamente la componente de luminancia Y del flujo YCbCr 4:2:2 (los bytes Cb y Cr se descartan), obteniendo una imagen en escala de grises.
- Insertar los marcadores de trama AA 55 AA 55 al inicio de cada frame para que el *SW* del PC pueda sincronizar el flujo de imágenes.

- Aplicar el protocolo de sustitución de bytes reservados: los valores 0xFF, 0x00, 0xAA y 0x55 que puedan aparecer en los datos de imagen se sustituyen por 0xFE, 0x01, 0xAB y 0x56 respectivamente, para evitar falsos positivos en la detección del marcador de inicio de trama. Cabe destacar que el MT9V111 limita su rango de salida activo aproximadamente al intervalo 0x10–0xEF, dejando fuera los valores extremos 0x00 y 0xFF por diseño interno del sensor; por tanto, en la práctica solo 0xAA y 0x55 pueden aparecer como valor de píxel real, mientras que la sustitución de 0xFF y 0x00 actúa como margen de seguridad adicional frente a posibles configuraciones futuras del sensor o valores fuera de rango nominal. Esta sustitución es inapreciable para el ojo humano y no afecta a los algoritmos de detección de rostros.
- Controlar la tasa de captura mediante el genérico g_TARGET_FPS, descartando frames completos cuando la tasa real supera la tasa objetivo. En la implementación final esta funcionalidad está configurada para no descartar frames.

La FIFO asíncrona (fifo_generator_0) se configura en modo [FWFT](#) (First Word Fall Through), lo que permite al módulo ftdi_controller leer el primer dato disponible sin necesidad de activar previamente rd_en: el dato ya está presente en dout en cuanto empty='0', lo que simplifica la lógica de lectura y elimina un ciclo de latencia por transacción.

6.2.4 FTDI Controller

El módulo ftdi_controller.vhd implementa el controlador del chip FT232H en modo FIFO Síncrona. Opera en el dominio ftdi_clk (60 MHz, CLKOUT del FT232H). Dado que TX e RX requieren flancos de muestreo distintos, el módulo se implementa con **dos procesos independientes**:

PROCESO TX (p_tx, falling_edge) El proceso p_tx gestiona la transmisión de imagen FPGA→PC. Opera en flanco de bajada de ftdi_clk, de modo que WR# y ADBUS[7:0] sean estables en el flanco de subida en que el FT232H los muestrea. El proceso está inhibido mientras s_rx_active='1' (el proceso RX está ocupando el bus). La elección del flanco de bajada ha sido **crucial e indispensable** en la implementación del protocolo de FIFO síncrona, no funcionando en este modo de elegir el flanco de subida por restricciones de tiempo.

PROCESO RX (p_rx, rising_edge) El proceso p_rx gestiona la recepción de comandos PC→FPGA. Opera en flanco de subida, ya que el FT232H presenta los datos en ADBUS[7:0] estables en el flanco de subida de CLKOUT (Figura 4.4 del datasheet [1]).

La secuencia de lectura sigue el protocolo de ráfaga continua requerido por el FT232H (AN130 [8]), representada en detalle en la Figura B.3 del Anexo B:

1. **ST_RX_PRE**: La FPGA desactiva adbus_oe (suelta el bus) con OE#='1' aún activo, eliminando la posibilidad de contención de bus.
2. **ST_RX_OE**: Se activa OE#='0': el FT232H comienza a conducir ADBUS.

3. **ST_RX_OE2**: Ciclo de margen ($OE\# = '0'$ durante al menos un ciclo completo antes de $RD\# = '0'$, como exige el datasheet).
4. **ST_RD0**: Se activa $RD\# = '0'$ (inicio de ráfaga). Ciclo vacío de pipeline: `adbus_i` puede no estar estable aún.
5. **ST_RD1...ST_RD6**: $RD\#$ permanece bajo. En cada `rising_edge` el FT232H avanza al siguiente byte de su FIFO interna. El byte leído en el estado RDx es el que el FT232H presentó durante el estado $RD(x-1)$, lo que introduce un pipeline natural de un ciclo.
6. **ST_RX_RELEASE / ST_RX_RELEASE2**: Se desactivan $RD\#$, $OE\#$ y `adbus_oe`. Dos ciclos de margen para estabilización.
7. **ST_RX_EXEC**: Se emite el pulso `cmd_valid_o` (1 ciclo) con el comando decodificado.

La señal `s_rx_active` actúa como handshake entre ambos procesos: se activa al inicio de la recepción y se desactiva en **ST_RX_EXEC**, asegurando que `p_tx` no conduce el bus durante toda la secuencia de lectura.

6.2.5 CMD Processor

El módulo `cmd_processor.vhd` recibe los comandos decodificados por `ftdi_controller` (dominio `ftdi_clk`) y los sincroniza al dominio `s_mclk` (100 MHz) mediante un sincronizador de doble flip-flop (`p_cdc`):

- `cmd_valid_i` se sincroniza con 3 FF (`ASYNC_REG`) y se detecta el flanco de subida mediante `s_cmd_pulse <= s_valid_sync1 AND NOT s_valid_sync2`.
- `cmd_type_i` y `cmd_data_i` se sincronizan con 2 FF cada uno.

El cruce **CDC** es de 60 MHz (origen) a 100 MHz (destino), es decir, el reloj destino es más rápido que el origen. El pulso `cmd_valid_o` de `ftdi_controller` dura 1 ciclo de `ftdi_clk` (16,67 ns), lo que garantiza al menos un flanco de `s_mclk` dentro del pulso. El valid usa una cadena de 3 FF: dos para sincronizar (2FF clásico) y un tercero para la detección de flanco (`sync1 AND NOT sync2`), que es lo que convierte el nivel sincronizado en un pulso de un ciclo.

El proceso `p_dispatch` ejecuta la acción correspondiente a cada comando:

- **CMD 0x01**: conmuta `s_led_toggle_r` (pulso de 1 ciclo que la lógica exterior usa para alternar los LEDs).
- **CMD 0x02**: registra `s_bcd_r` con el valor de `DATA[15:0]`.
- **CMD 0x03**: lanza una transacción de escritura **I2C** a través de `p_exec`.
- **CMD 0x04**: registra `s_cap_en_r` con `DATA[0]`.
- **CMD 0x05**: registra `s_sim_en_r` con `DATA[0]`.

Todas las señales de salida del `cmd_processor` (`cap_en_cmd_o`, `sim_img_en_o`) que cruzan al dominio `mt_pixclk_i` se resincronización en `TOP.vhd` mediante sincronizadores 2FF adicionales, marcadas con el atributo `ASYNC_REG`.

El comando `CMD 0x03` (I2C) solo se atiende cuando la señal `i2c_grant_i` está activa, lo cual en `TOP.vhd` únicamente ocurre con `s_state = ST_FINISH` (es decir, una vez completada la configuración inicial del sensor descrita en la Sección 6.2.1). Si llega un comando I2C mientras el sistema todavía está en la secuencia de configuración, o mientras ya hay una escritura I2C pendiente de ejecutar, se descarta silenciosamente (sin notificaciones de ningún tipo). Esto es relevante para la Prueba P4 (Tabla 9.4): los comandos de escritura de registro deben enviarse una vez el sistema ha terminado de arrancar.

6.2.6 Imagen sintética (*cam_sim*)

El módulo `mt9v111_image.vhd` genera un patrón de imagen sintético que emula el protocolo de salida del sensor MT9V111 (señales `fvalid`, `lvalid`, `data` y `pixclk`). Se instancia en `TOP.vhd` y se *clockea* con `mt_pixclk_i` (mismo dominio que `frame_capture`), lo que evita introducir BUFGMUX adicionales y mantiene intacto el CDC.

La selección entre imagen real (sensor) e imagen sintética se realiza mediante un multiplexor combinacional en `TOP.vhd`, controlado por la señal `s_use_sim_image_pix` (sincronizada al dominio `mt_pixclk_i`). Dicha señal no se fija en tiempo de síntesis, sino que se controla en tiempo de ejecución mediante un comando enviado desde el PC: el `cmd_processor` decodifica el comando correspondiente y actualiza el registro `s_sim_en_r`, permitiendo conmutar entre el sensor físico y la imagen sintética sin resintetizar el diseño. El valor por defecto tras reset es imagen sintética (`s_sim_en_r <= '1'`), lo que permite validar la cadena de transmisión sin necesidad del sensor físico.

6.3 GESTIÓN DE CRUCES DE DOMINIO DE RELOJ (CDC)

El diseño presenta tres cruces de dominio de reloj, resumidos en la Tabla 6.2.

Tabla 6.2: Cruces de dominio de reloj en el diseño VICON.

Señal	Origen	Destino	Solución
<code>cmd_valid</code> , <code>cmd_type</code> , <code>cmd_data</code> , <code>cmd_addr</code> , <code>cmd_page</code>	<code>ftdi_clk</code> (60 MHz)	<code>s_mclk</code> (100 MHz)	2FF (datos) + 3FF con detección de flanco (<code>valid</code>)
<code>s_cap_en</code>	<code>s_mclk</code> (100 MHz)	<code>mt_pixclk_i</code> (25 MHz)	2FF + <code>ASYNC_REG</code>
<code>s_use_sim_image</code>	<code>s_mclk</code> (100 MHz)	<code>mt_pixclk_i</code> (25 MHz)	2FF + <code>ASYNC_REG</code>

El cruce `ftdi_clk`→`s_mclk`, correspondiente a la recepción de comandos, se resuelve con sincronizadores en el `cmd_processor`: una cadena de dos biestables (2FF) para los campos de datos del comando y una cadena de tres biestables para la señal de validez,

cuyos dos últimos registros permiten detectar el flanco de subida y generar un pulso único de comando nuevo. La primera etapa de estos sincronizadores se excluye del análisis de *timing* entre dominios mediante `set_false_path`, ya que la metaestabilidad la resuelve la propia cadena de sincronización.

Los cruces `s_mclk→mt_pixclk_i` (`s_cap_en` y `s_use_sim_image`) son señales de nivel generadas a partir de comandos recibidos por USB y mantenidas estables hasta la recepción de un nuevo comando, es decir, con una duración mínima del orden de milisegundos, muy superior al periodo de `mt_pixclk_i` (≈ 37 ns a 27 MHz). Al tratarse de dominios de reloj rápido (100 MHz) a lento (27 MHz), un cruce mediante sincronizador doble solo es seguro si la señal de origen no puede perderse entre dos muestreos consecutivos del dominio destino; dado que aquí el nivel permanece estable durante varios órdenes de magnitud más que un periodo de `mt_pixclk_i`, no existe riesgo de que el sincronizador "salte" el cambio, y por tanto no es necesario un protocolo de petición/confirmación (*handshake*) adicional. Los biestables dobles se implementan con el atributo `ASYNC_REG`, que fuerza a la herramienta a ubicarlos físicamente próximos entre sí para maximizar el tiempo disponible frente a la metaestabilidad.

Por último, tanto el reloj `mt_pixclk_i` como el reloj `ftdi_clk` (CLKOUT del FT232H) se introducen en la [FPGA](#) a través de un IBUF con la propiedad `CLOCK_DEDICATED_ROUTE FALSE`, dado que los pines físicos empleados no son pines [CCIO](#) (Clock Capable Input/Output) en la Artix-7 de la Basys 3.

6.4 VERIFICACIÓN POR SIMULACIÓN

6.4.1 Estructura del banco de pruebas

El [TB](#) (TestBench) se organiza en los siguientes componentes:

- **testbench.vhd**: instancia el [DUT](#) (Device Under Test) (T0P), los agentes de verificación, los generadores de reloj y los monitores. Actúa como *test harness*.
- **clk_reset_gen.vhd**: genera el reloj base de 100 MHz y la señal de reset activo en alto.
- **mt9v111_i2c.vhd**: agente esclavo [I2C](#) que simula el comportamiento del sensor MT9V111, respondiendo a las transacciones de escritura ([ACK](#) (Acknowledge)) y de lectura (devuelve datos de registro predefinidos).
- **ftdi_agent.vhd**: agente que simula el chip FT232H, incluyendo el generador de CLKOUT (60 MHz), la FIFO de recepción de imagen y el inyector de comandos PC→FPGA.
- **mt9v111_image.vhd**: generador de imagen sintética reutilizado directamente del diseño [RTL](#).

La Figura [6.3](#) muestra el diagrama de bloques del banco de pruebas, con las señales físicas que conectan el [DUT](#) con cada agente. Nótese que `mt9v111_image.vhd` no se instancia de forma independiente en `testbench.vhd`: al fijarse `g_USE_CAM_SIM => TRUE`, es el propio [DUT](#) quien lo instancia internamente (`u_cam_sim`), por lo que las entradas

mt_data_i, mt_fvalid_i y mt_lvalid_i del DUT permanecen sin uso durante toda la simulación.

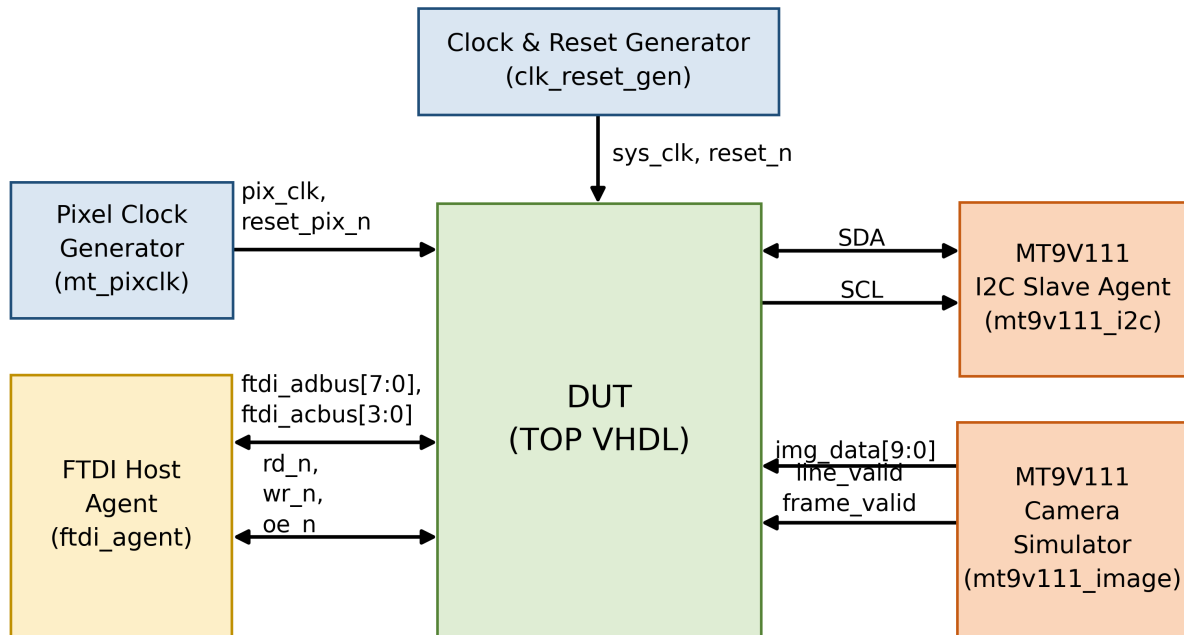


Figura 6.3: Diagrama de bloques del banco de pruebas de VICON.

Las librería UVVM utilizada es la uvvm_util (logging, alertas, check_value).

6.4.2 Agente FTDI

El agente ftdi_agent.vhd modela el comportamiento del FT232H en modo *Synchronous FIFO* con especial atención al protocolo de ráfaga continua validado en hardware:

- **Inyección de comandos:** la tabla c_CMD_TABLE define los comandos a inyectar con sus bytes serializados y su longitud (4 o 6 bytes). El proceso p_tx_inject activa RXF#='0' y presenta los bytes en adbus_io de forma combinacional cuando OE#='0'. Cada rising_edge con OE#='0' y RD#='0' consume un byte y el agente avanza al siguiente, sin subir RD# entre bytes.
- **Captura de imagen:** el proceso p_rx registra cada byte presentado en ADBUS cuando WR#='0' en rising_edge, almacenándolo en una FIFO circular.
- **Control de TXE#:** el proceso p_txe activa TXE#='1' (buffer lleno) cuando el nivel de la FIFO supera g_TXE_BUSY_THRESHOLD, modelando la contrapresión del chip real.

6.4.3 Monitores

El testbench incluye tres monitores pasivos implementados como procesos VHDL:

- **p_monitor**: observa la señal `a_fsm_state` (alias externo VHDL-2008 hacia `u_dut.s_state`) y loguea cada transición de estado de la FSM de configuración, incluyendo el valor de `a_rd_data` en los estados de verificación de registros.
- **p_frame_monitor**: cuenta los flancos de subida de `a_frame_done` y loguea el número de frame completado.
- **p_i2c_sniffer**: decodifica bit a bit el tráfico del bus I2C, identificando condiciones START, STOP, RSTART, bytes de dirección y datos con sus ACK/NACK.

Los alias de señales externas («`SIGNAL u_dut.s_state : main_state_t`») se compilan con la opción `-suppress 1309` en ModelSim/QuartaSim para suprimir la advertencia de elaboración generada por señales de tipo enumerado accedidas desde fuera de la arquitectura.

6.4.4 Resultados de simulación

Además de los monitores descritos en el apartado anterior, el propio `testbench.vhd` automatiza una comprobación de aceptación de la simulación. El proceso `p_monitor` recorre cronológicamente cada estado de la FSM de configuración (Sección 6.2.1), logueando cada transición; al alcanzar `ST_FINISH` espera dos flancos de `s_clk_base` y verifica mediante `check_value` que el LED 0 de la Basys 3 está a nivel alto, confirmando de forma automática y repetible que la configuración del sensor ha finalizado sin errores. Si la FSM alcanza en cambio `ST_ERROR`, el mismo proceso lanza una alerta de UVVM indicando el índice de `c_CFG_TABLE` y el valor leído que provocó el fallo.

En paralelo, `p_frame_monitor` cuenta y loguea cada fotograma completado (flanco de subida de `frame_done_o`), y `p_i2c_sniffer` decodifica de forma independiente, bit a bit, el tráfico del bus I2C (condiciones START/RSTART/STOP, direcciones y bytes con su ACK/NACK), actuando como comprobación cruzada frente al propio `i2c_controller` bajo verificación. El DUT se instancia con `g_USE_CAM_SIM => TRUE` (la imagen procede del generador sintético interno `mt9v111_image.vhd`, no de un sensor físico) y con el ILA deshabilitado (`g_USE_ILA => FALSE`); el reloj de píxel (`mt_pixclk_i`, 25 MHz) se genera de forma independiente en el propio `testbench`, ya que en esta arquitectura ese reloj lo aporta siempre el sensor —real o simulado—, nunca el sistema.

La Figura 6.4 recoge una vista general de la simulación completa. En la parte superior (señales `s_basys3_led/s_basys3_cat`) se aprecian los cambios provocados por los comandos que el agente `ftdi_agent` inyecta según `c_CMD_TABLE` (conmutación de LEDs y escritura de un registro I2C) una vez el sistema ya está configurado; en el grupo FSM/I2C se observa la ráfaga de transacciones I2C de configuración completándose hasta alcanzar `ST_FINISH`; y en el grupo frame capture se ve cómo, una vez habilitada la captura (`s_cap_en`), el módulo `frame_capture` comienza a escribir en la FIFO asíncrona los bytes de la imagen sintética (FIFO DATA).

La Figura 6.5 muestra en detalle la actividad del generador de imagen sintética a través de los contadores internos de `frame_capture` (FSM STATE, BYTE CNT, COL CNT, ROW CNT). Cada pulso de `LINE_VALID` hace avanzar el contador de columna, que se reinicia al llegar al ancho de línea configurado para la simulación, incrementando entonces el contador de fila; `FRAME_VALID` delimita a su vez el fotograma completo. A

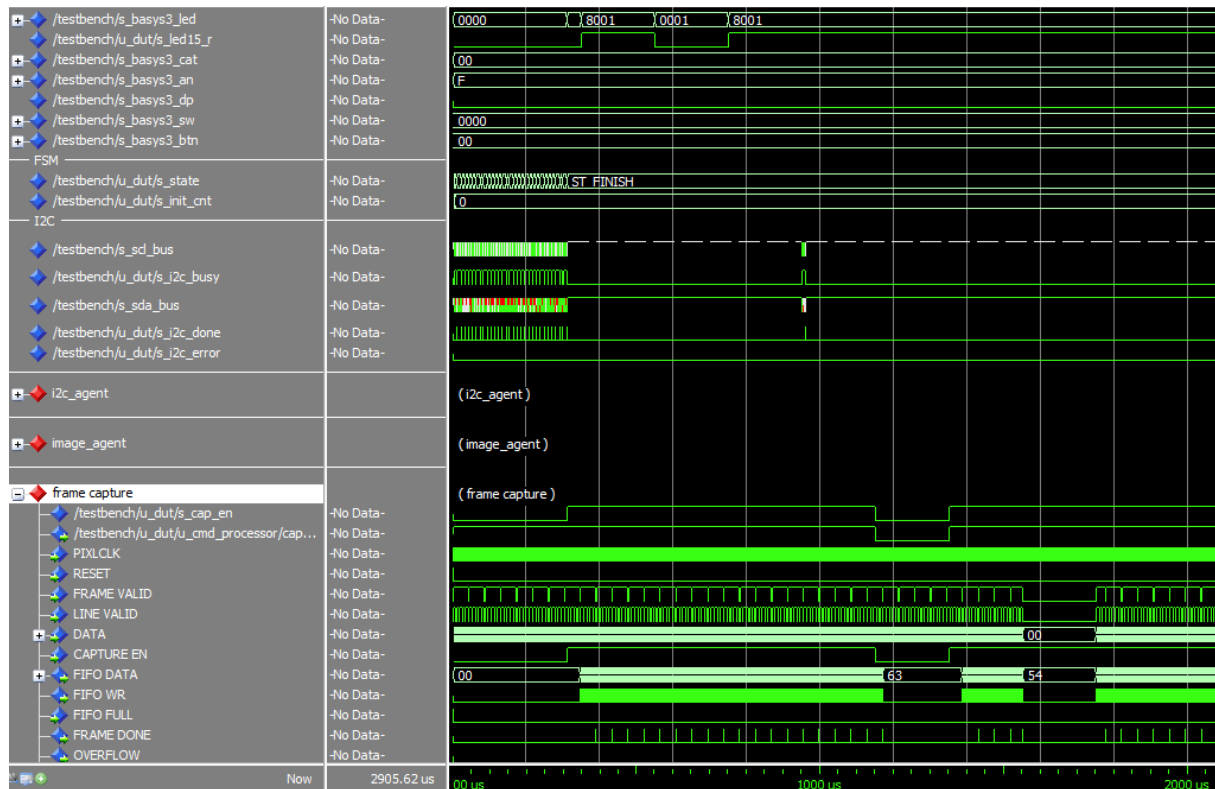


Figura 6.4: Vista general de la simulación: configuración I2C, inicio de captura y ejecución de comandos.

diferencia de la resolución real del sensor (640×480), en simulación las dimensiones de la imagen sintética (`g_CAM_SIM_H_RES/g_CAM_SIM_V_RES`) se configuran mucho más reducidas —de ahí que los contadores de columna y fila se reinicien cada pocos ciclos—, precisamente para acelerar la simulación sin necesidad de recorrer un fotograma completo de 640×480 píxeles en cada ejecución del testbench.

Por último, la Figura 6.6 muestra la recepción de un comando de LED (`CMD 0x01`) inyectado por el agente FTDI. En el grupo `FTDI_CTRL` se observa la trama `0xCC` (byte de sincronismo) seguida de los bytes del comando en `adbus_i`; en el grupo `CMD_PROCESSOR`, la cadena de sincronización `s_type_sync0/s_type_sync1` confirma el valor `0x01` ya cruzado al dominio `s_mclk`, el pulso `s_cmd_pulse` marca el ciclo en el que se detecta un comando nuevo (Sección 6.2.5), y `s_exec_state` sale momentáneamente de `ST_EXEC_IDLE` para realizar la acción. El nivel de la FIFO de imagen del propio agente (`rx_fifo_level`) continúa decreciendo durante la misma ventana, mostrando que la transmisión de vídeo prosigue de forma intercalada con la atención al comando entrante.

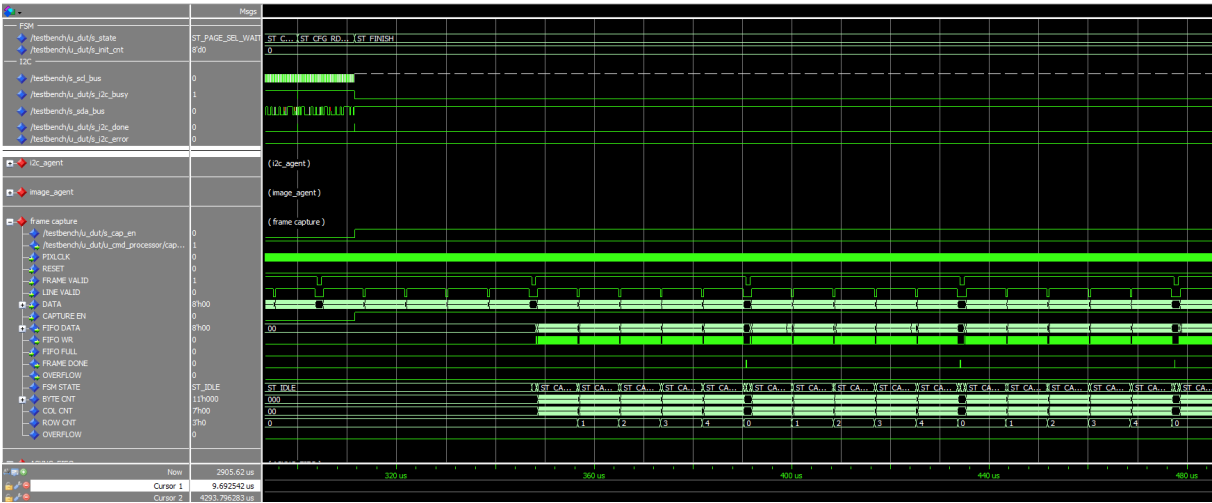


Figura 6.5: Detalle del generador de imagen sintética: contadores de columna y fila de frame_ - capture.

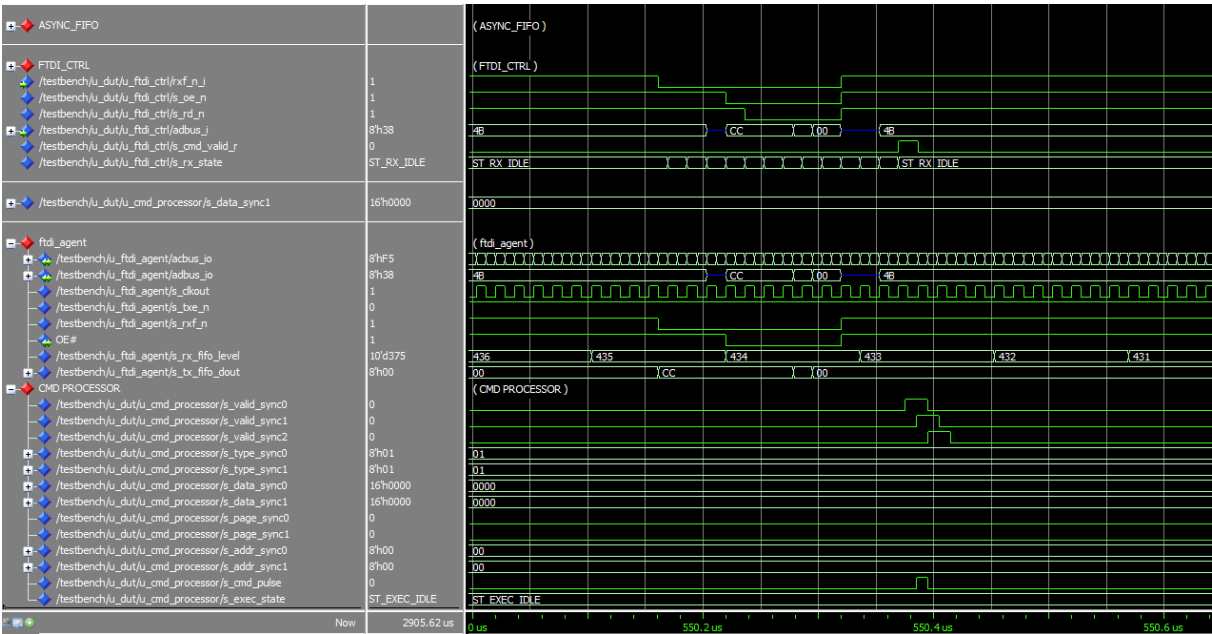


Figura 6.6: Recepción y decodificación de un comando LED (CMD 0x01) durante la simulación.

SOFTWARE DE VISUALIZACIÓN PC

La aplicación `vicon_view` es un programa de escritorio desarrollado en C++ con Qt 6 (interfaz gráfica) y OpenCV, este último para la detección de rostros. Se comunica con la FPGA a través del chip FT232H mediante la API oficial de FTDI (FTD2XX), recibiendo el flujo de imagen en escala de grises y permitiendo el envío de comandos de control. El proyecto se compila con CMake (`CMakeLists.txt`), para la generación automática de los ficheros de Qt a partir de `mainwindow.ui` y `resources.qrc`.

La elección de C++ para el software de PC no fue casual o basada únicamente en el requisito, sino consecuencia directa de un problema detectado en una primera versión desarrollada en Python. Con Python, la recepción del vídeo fallaba de forma intermitente: la **FIFO** asíncrona de la **FPGA** se desbordaba (overflow) y se perdía el sincronismo de trama.

El diagnóstico con ILA reveló que la causa no era el ancho de banda —el enlace FT232H ofrece hasta 60 MB/s— sino la **latencia del host** en la lectura del buffer. El flujo de imagen es un caudal continuo: mientras el PC no vacía el buffer del FT232H, la **FPGA** sigue escribiendo en la **FIFO** asíncrona, que tiene una capacidad limitada. Si el PC deja de leer durante un intervalo demasiado largo, la **FIFO** se llena y se produce el desbordamiento.

En Python, estos intervalos resultaban excesivos e impredecibles: se introducían pausas de varios milisegundos en la ejecución, durante las cuales no se leía del FT232H. Se midieron latencias de lectura del orden de 7 ms, cuando la capacidad de la **FIFO** solo permitía tolerar pausas de aproximadamente 2,6 ms antes de desbordar. Es decir, las pausas superaban en más del doble el margen disponible.

La migración a C++ resolvió el problema. Al ejecutar la recepción en un hilo dedicado con alta prioridad, las latencias de lectura se reducen notablemente, dentro del margen que permite la **FIFO**. Esto permite mantener la máxima tasa de fotogramas sin necesidad de recurrir a soluciones como reducir los **FPS** o sobredimensionar la **FIFO**, y confirma que el cuello de botella residía por completo en el lado del PC, no en la **FPGA**.

7.1 ARQUITECTURA Y CONCURRENCIA

A diferencia de versiones anteriores basadas en una única llamada bloqueante a `FT_Read` en el hilo principal (lo que llegaba a congelar la interfaz), la aplicación actual separa el trabajo en **tres hilos** independientes, comunicados con la **GUI** exclusivamente mediante señales y `slots` de Qt (nunca actualizando directamente un widget desde un hilo secundario):

- **Hilo principal** (`QApplication::exec()`): ejecuta el bucle de eventos de Qt, gestiona los *widgets* generados desde `mainwindow.ui` y actualiza la imagen, el indicador de FPS y el estado de conexión en los *slots* `onFrameReady`, `onConnectionStatusChanged` y `onFacesDetected`.
- **Hilo de captura** (`captureThreadFunc`): en Windows se eleva a prioridad `THREAD_PRIORITY_TIME_CRITICAL`. Llama a `FT_Read` de forma directa (sin mutex, ya que es el único hilo que lee del dispositivo), acumula los bytes recibidos, localiza el marcador de trama y emite la señal `frameReady(QImage, fps)` por cada fotograma completo.
- **Hilo de detección de rostros** (`detectionThreadFunc`): coge el último fotograma disponible (compartido mediante `m_latestFrame` protegido por `m_latestFrameMutex`), lanza el clasificador Haar de OpenCV y emite la señal `facesDetected(QVector<QRect>, detFps)` con los rectángulos detectados.

La sincronización entre hilos se apoya en tres objetos distintos, cada una con un propósito concreto:

- `m_ftdiMutex`: protege únicamente las llamadas a `FT_Write` (comandos salientes desde el hilo principal), que pueden solaparse en el tiempo con la lectura continua del hilo de captura sobre el mismo *handle* FTDI.
- `m_latestFrameMutex`: protege el traspaso del último fotograma capturado desde el hilo de captura hacia el hilo de detección.
- `m_facesMutex`: protege la lista de rectángulos detectados, consumida por el hilo principal al pintar el fotograma.

Cuando no hay datos disponibles (buffer vacío en captura, o detección desactivada), ambos hilos secundarios recurren a una espera fija breve (`std::this_thread::sleep_for`, entre 1 y 30 ms según el caso) en lugar de una variable de condición; es una solución simple, aunque una futura revisión podría sustituirla por primitivas de sincronización basadas en eventos si se necesitara reducir aún más la latencia.

7.2 INTERFAZ GRÁFICA

La interfaz (`mainwindow.ui`) se organiza en dos columnas dentro de un `QHBoxLayout`, tal y como puede verse en la Figura A.1 (Apéndice A):

- **Columna izquierda**: la ventana de vídeo (`lblVideo`), con el indicador de FPS de captura (`lblFpsRx`) y el estado de conexión (`lblStatus`, en verde si el FT232H está abierto en modo *Sync FIFO* y en rojo si no).
- **Columna derecha**, organizada en `QGroupBox` independientes: *Cámara* (activar/desactivar captura y alternar entre sensor real e imagen sintética), *Detección de rostros*, *LEDs* (dos botones individuales), *Configuración (I2C)* y *Log de comandos*.

Dos botones adicionales (*Enviar Comando RAW...* y *Enviar Comando I2C...*) se añaden de forma **dinámica** al grupo de *Configuración* desde el propio constructor de MainWindow (no están definidos en el `.ui`), y abren un `QInputDialog` para introducir el comando en texto libre.

7.3 PROTOCOLO DE RECEPCIÓN DE IMAGEN

La función `findMarker` busca en el flujo de bytes acumulado la secuencia `0xAA 0x55 0xAA 0x55` que marca el inicio de cada fotograma. Una vez localizado el marcador, se descartan los bytes previos y se extraen exactamente `kFrameBytes = 640 × 480 = 307 200` bytes de luminancia. A diferencia de reconstruir la imagen RGB duplicando el valor de luminancia en los tres canales ($R=G=B=Y$), la implementación actual usa directamente el formato nativo `QImage::Format_Grayscale8`, evitando la copia y conversión adicional. Si el acumulador de bytes crece por encima de $4 \times$ el tamaño de lectura (`kBufSize = 65 536` B) sin encontrar el marcador, se descarta todo el contenido salvo los 3 últimos bytes (para no perder un marcador que quede entre dos lecturas), limitando así el crecimiento de memoria ante una posible pérdida de sincronismo prolongada.

El indicador de FPS de captura (`lblFpsRx`) se calcula con una ventana deslizante de hasta 30 marcas de tiempo (`std::deque`) sobre los fotogramas efectivamente recibidos, no sobre los bytes leídos del USB.

7.4 PROTOCOLO DE COMANDOS Y CONTROL

Todos los comandos comparten el mismo byte de sincronismo (`kCmdSync = 0xCC`) y se serializan en tramas de 4 o 6 bytes mediante `sendCmd4/sendCmdI2C`, coherentes con el protocolo de `ftdi_controller` descrito en la Sección 6.2.4. La Tabla 7.1 resume los identificadores de comando definidos en el código (`kCmdLed`, `kCmdBcd`, `kCmdI2c`, `kCmdCap`, `kCmdSrc`), que coinciden con los `CMD 0x01–0x05` del `cmd_processor` (Sección 6.2.5).

Tabla 7.1: Comandos PC→FPGA definidos en `mainwindow.cpp`.

Constante	Valor	Origen en la GUI
<code>kCmdCap</code>	<code>0x04</code>	Botón <i>Deshabilitar/Habilitar cámara</i> (<code>onToggleCamera</code>).
<code>kCmdSrc</code>	<code>0x05</code>	Botón <i>Fuente: Real / Modo: Sintético</i> (<code>onToggleImageSource</code>).
<code>kCmdLed</code>	<code>0x01</code>	Botones <i>LED 0/LED 1</i> (<code>onLed0/onLed1</code>).
<code>kCmdI2c</code>	<code>0x03</code>	Diálogo <i>Enviar Comando I2C...</i> (<code>onSendI2CCommand</code>), trama de 6 bytes con página, registro y dato.

Los botones *LED 0*/*LED 1* envían en el campo de datos una máscara combinada de 2 bits (bit0=estado de LED 0, bit1=estado de LED 1) mediante `kCmdLed`. Sin embargo, en el procesador de comandos actualmente se interpreta como un **pulso de conmutación** (`s_led_toggle_r`) que alterna el LED de la FPGA, no como una escritura directa de una máscara de valor.

Además de los comandos con botón propio, dos diálogos genéricos permiten enviar tramas arbitrarias sin recompilar la aplicación:

- **Comando RAW** (`onSendRawCommand`): solicita una lista de bytes en hexadecimal o decimal separados por espacios y los transmite tal cual con `FT_Write`, validando que cada valor esté en el rango de 8 bits.
- **Comando I2C** (`onSendI2CCommand`): solicita página, dirección de registro y dato (p. ej. 1 0x37 0x0080 para forzar los 30 fps del registro R55 descrito en la Sección 6.2.1), valida los rangos (página/registro de 8 bits, dato de 16 bits) y compone la trama de 6 bytes correspondiente. Esta es la forma utilizada en la Figura A.2 del Apéndice A.

7.5 DETECCIÓN DE ROSTROS

Al arrancar, `MainWindow` intenta cargar el clasificador Haar `haarcascade_frontalface_default.xml` desde el directorio del ejecutable (`QCoreApplication::applicationDirPath()`); si falla, la detección queda deshabilitada y el botón correspondiente lo indica en la barra de estado. Cuando está activa, el hilo de detección ejecuta `cv::CascadeClassifier::detectMultiScale` sobre el último fotograma en escala de grises (factor de escala 1.2, mínimo 4 vecinos, tamaño mínimo de detección 40×40 px) y los rectángulos resultantes se dibujan en rojo sobre el `QPixmap` mostrado, en el hilo principal, dentro de `onFrameReady`.

7.6 GESTIÓN DE ERRORES Y LIMITACIONES

El principal problema de la arquitectura original —el bloqueo de la interfaz gráfica mientras `FT_Read` esperaba datos— queda resuelto en la versión actual al ejecutar la captura en su propio hilo y comunicar los resultados a la GUI exclusivamente mediante señales de Qt, sin llamadas bloqueantes en el hilo principal. Las limitaciones que persisten son de menor importancia:

- `CMakeLists.txt` fija `WIN32_EXECUTABLE TRUE` en Windows, por lo que la aplicación no abre consola: cualquier traza de depuración enviada a `stdout/stderr` queda oculta.
- Si `FT_Open` falla al arrancar, la aplicación se limita a mostrar un mensaje único en la barra de estado y continúa sin conexión; no existe una lógica de reintento ni de reconexión en caliente si el cable USB se desconecta y se vuelve a conectar durante la ejecución.
- Las esperas de los hilos de captura y detección cuando no hay datos disponibles usan temporizaciones fijas (`sleep_for` de 1–30 ms) en lugar de variables

de condición; funcional pero mejorable si se quisiera minimizar la latencia al máximo.

IMPLEMENTACIÓN Y RESULTADOS

Este capítulo detalla el proceso de implementación del diseño sobre la Basys 3, incluyendo los resultados de síntesis e implementación en Vivado, las restricciones de temporización y de pines empleadas, el proceso de depuración en hardware apoyado en el [ILA](#) y, finalmente, las pruebas realizadas sobre la placa física.

8.1 SÍNTESIS E IMPLEMENTACIÓN EN VIVADO

El proyecto se sintetiza e implementa con Vivado 2020.2 sobre el dispositivo (target) XC7A35T-1CPG236C.

8.1.1 Resultados de timing

El análisis de tiempos tras la implementación (`report_timing_summary`) revela una violación de *setup* localizada exclusivamente en la interfaz de salida hacia el FT232H, tal como resume la Tabla 8.1. El resto del diseño (dominios `s_mclk` y `mt_pixclk_i`, y las trayectorias de *hold*) cierra sin violaciones.

Tabla 8.1: Resumen de *timing* tras la implementación.

Métrica	Valor	Resultado
WNS (<i>Worst Negative Slack</i>)	−9,766 ns	VIOLADO
TNS (<i>Total Negative Slack</i>)	−94,162 ns	VIOLADO
Endpoints con violación	21 / 6349	—
WHS (<i>Worst Hold Slack</i>)	+0,056 ns	OK
WPWS (<i>Worst Pulse Width Slack</i>)	+3,000 ns	OK

Todos los *endpoints* en violación pertenecen al mismo grupo: las señales que la [FPGA](#) escribe hacia el FT232H (`WR#`, `RD#`, `OE#` y el bus `ADBUS[7:0]`), gobernadas por el reloj `ftdi_clk`. El desglose del peor camino (Tabla 8.2) permite identificar la causa raíz.

Tabla 8.2: Desglose del peor camino ($s_wr_n_reg \rightarrow WR\#$).

Contribución	Valor (ns)	Origen
Requisito (medio periodo)	8,333	TX en flanco de bajada
Inserción de reloj (CLKOUT)	−5,422	pin P17 no <i>clock-capable</i>
Retardo del camino de datos	−4,992	FF + OBUF
<i>Output delay</i> (setup FT232H)	−7,650	$t_{su} = 7,5 \text{ ns} + \text{skew PCB}$
Slack resultante	−9,766	

La causa no reside en la lógica del diseño —el camino de datos tiene un único nivel lógico (OBUF) y tarda menos de 5 ns—, sino en la **inserción del reloj CLKOUT**. Este reloj de 60 MHz, generado por el FT232H, entra por el conector *Pmod* en el pin P17, que **no es un pin *clock-capable***. En la placa Basys 3, ningún pin de los conectores *Pmod* dispone de acceso a la red de reloj dedicada; el único pin *clock-capable* disponible para el usuario es el del oscilador de la placa. Por esta razón, Vivado se ve obligado a rutar CLKOUT hasta el BUFG a través del *fabric* de rutado general, introduciendo un retardo de inserción de 5,422 ns. Sumado al requisito de *setup* del FT232H (7,5 ns), este retardo consume por completo el medio periodo disponible entre el flanco de lanzamiento y el de captura, lo que da lugar al *slack* negativo.

Esta limitación es de naturaleza física y no puede resolverse mediante restricciones ni reorganización del RTL: la única solución que la eliminaría por completo sería reubicar CLKOUT en un pin *clock-capable*, opción inviable al estar la conexión del módulo FT232H fijada por el diseño de la placa adaptadora, o bien migrar el FT232H al modo *Asynchronous FIFO*, que prescinde del reloj externo a costa de un menor ancho de banda.

No obstante, el sistema opera de forma correcta y estable sobre Hardware. La violación indicada por el análisis estático corresponde al peor caso. En las condiciones reales de operación, con cables de conexión cortos y de igual longitud entre CLKOUT y el bus de datos, los tiempos se mantienen dentro de los márgenes que el FT232H requiere, como confirma la recepción íntegra del vídeo en el PC durante la validación (Capítulo 9). Se trata, por tanto, de un caso en el que el cumplimiento estricto del análisis estático de tiempos está limitado por la plataforma, pero la funcionalidad queda garantizada por medio de las pruebas realizadas.

8.1.2 Utilización de recursos

La Tabla 8.3 recoge la ocupación del diseño sobre la **FPGA** Artix-7 XC7A35T tras la implementación. Se distingue entre la ocupación total del *bitstream* de depuración y la del sistema propiamente dicho, ya que el núcleo de depuración (*dbg_hub*, asociado al **ILA**) se retira en la configuración final y no debe contabilizarse como coste del diseño.

Tabla 8.3: Utilización de recursos en la XC7A35T.

Recurso	Total	Sin ILA	Disp.	% (sin ILA)
LUT (<i>Slice LUTs</i>)	2003	1555	20 800	7,5 %
FF (<i>Slice Registers</i>)	3102	2361	41 600	5,7 %
BRAM (<i>Block RAM Tile</i>)	7,5	7,5	50	15,0 %
DSP	0	0	90	0,0 %
MMCM	1	1	5	20,0 %
BUFG	6	6	32	18,8 %
IOB (<i>Bonded IOB</i>)	74	74	106	69,8 %

La ocupación lógica del diseño es reducida —en torno al 7,5 % de las **LUT** (Look Up Table)s y al 5,7 % de los biestables—, lo que indica que la **FPGA** seleccionada ofrece un margen amplio para futuras ampliaciones, como el procesamiento de imagen en el propio hardware. El recurso más comprometido es el de pines de entrada/salida (**IOB** (Input/Output Block)), con casi un 70 % de ocupación, debido al elevado número de señales que exponen la interfaz del sensor y el bus del FT232H hacia los conectores *Pmod*.

8.2 CONSTRAINTS XDC

El fichero de constraints `Basys3_GPIO.xdc` define las siguientes restricciones de timing:

- **Relojes:** `create_clock` para `basys3_clk_i` (100 MHz), `ftdi_clk` (60 MHz) y `mt_pixclk_i` (25 MHz).
- **Aislamiento de dominios asíncronos:** `set_clock_groups -asynchronous` entre los grupos `{clk_out1_clk_wiz_0, clk_out3_clk_wiz_0}` y `{ftdi_clk}`, y entre `{clk_out1}` y `{mt_pixclk_i}`.
- **False paths CDC:** `set_false_path` desde `ftdi_clk` hacia los registros de sincronización `s_cmd_valid_r_reg`, `cmd_type_o_reg` y `cmd_data_o_reg`.
- **Routing:** `CLOCK_DEDICATED_ROUTE FALSE` en el neto `mt_pixclk_i_IBUF`, dado que el pin físico utilizado no es un pin de reloj dedicado (CCIO) de la Artix-7.

8.3 DEPURACIÓN EN HARDWARE CON ILA

Durante el desarrollo se emplearon numerosos *cores* ILA (*Integrated Logic Analyzer*) para depurar sobre hardware las distintas etapas del diseño —la configuración del sensor por I2C, la captura de trama, el protocolo del FT232H o los cruces de dominio de reloj—, que se fueron retirando a medida que cada bloque quedaba validado. En la versión final del diseño se conservan únicamente dos, asociados a la depuración del procesador de comandos:

- **ila_0** (dominio `s_mclk`, 100 MHz): monitoriza el interior de `cmd_processor`, incluyendo los registros de salida (`s_led_toggle_r`, `s_cap_en_r`, `s_sim_en_r`), la cadena CDC (`s_valid_sync1`, `s_valid_sync2`, `s_cmd_pulse`) y el tipo de comando sincronizado (`s_type_sync1`).
- **ila_3** (dominio `s_mclk`, 100 MHz): monitoriza las entradas de `cmd_processor` y el estado de la FSM de ejecución I2C (`s_exec_state`).

Estos *cores* de depuración no forman parte de la funcionalidad del sistema y se eliminan en la configuración final, por lo que no deben contabilizarse como coste del diseño.

8.3.1 Diagnóstico del protocolo FT232H

El ILA `ila_0` fue determinante para identificar el bug de decodificación de comandos. Con trigger en `s_cmd_pulse='1'`, se observó que `s_type_sync1` mostraba continuamente el valor `0xCC` (marcador de sincronismo) en vez del tipo de comando real. Este patrón reveló que el módulo `ftdi_controller` estaba leyendo el mismo byte dos veces consecutivas en vez de avanzar al siguiente.

El análisis del timing del FT232H (Figura 4.4 del datasheet) mostró que el chip presenta los datos estables en el flanco de subida de `CLKOUT`, mientras que el diseño original muestreaba en `falling_edge`. La solución definitiva consistió en:

1. Separar TX (`falling_edge`) y RX (`rising_edge`) en dos procesos independientes.
2. Implementar el protocolo de ráfaga continua (`RD#` mantenido bajo durante todos los bytes) en vez de toggle por byte.
3. Introducir un ciclo de pipeline vacío (`ST_RD0`) para absorber el tiempo de presentación del primer byte.

8.4 PRUEBAS EN HARDWARE

8.4.1 Transmisión de imagen

Con la FPGA programada y el sensor conectado, `vicon_view` recibe y muestra correctamente el flujo de imagen en escala de grises a resolución VGA (640×480) a la tasa objetivo de 30 fps (configurada en `config_pkg.vhd` mediante `c_MT9V111_TARGET_FPS` y con la frecuencia fija del reloj generado de 30MHz), tal y como se describe en la Sección 6.2.1.

8.4.2 Comandos de control

Tras la corrección del protocolo FT232H, los comandos de control funcionan con un único envío:

- **CMD 0x01 (LED)**: el LED seleccionado se conmuta instantáneamente y el streaming de imagen continúa sin interrupción.

- CMD 0x03 (I2C): la escritura de registros del sensor se ejecuta correctamente, verificable tanto por el ILA como por el efecto visual en la imagen.
- CMD 0x04 (CAP): la desactivación de la captura detiene el streaming; la reactivación lo reanuda.
- CMD 0x05 (SIM): la conmutación entre imagen real e imagen sintética funciona correctamente.

VERIFICACIÓN Y VALIDACIÓN

Este capítulo presenta el proceso de verificación y validación del sistema **VICON** frente a los requisitos definidos en el Capítulo 4, describiendo tanto las pruebas realizadas como el plan de validación seguido para confirmar el correcto funcionamiento del sistema completo.

9.1 VERIFICACIÓN DE REQUISITOS

La verificación tiene por objeto comprobar que el sistema cumple el conjunto de requisitos especificados en el Capítulo 4. Siguiendo un enfoque textual, para cada prueba se define el requisito o requisitos que verifica (total o parcialmente), el entorno en el que se ejecuta, la descripción de la prueba, la salida esperada y los posibles errores. Algunas pruebas verifican varios requisitos a la vez y otras se centran en uno solo.

Antes de definir las pruebas, se describen los dos entornos en los que se ejecutan.

9.1.1 Entornos de prueba

ENTORNO DE SIMULACIÓN: Se dispone de un *Banco de pruebas* o **TB** de sistema, desarrollado con **UVVM** sobre QuestaSim, que verifica el diseño RTL completo sin necesidad de Hardware. El diseño se estimula con una imagen sintética y se conecta a un modelo del módulo FT232H. Este modelo es bidireccional, por lo que el *testbench* verifica tanto el camino de datos de vídeo (FPGA → PC) como la recepción de comandos (PC → FPGA). Permite comprobar de forma controlada y reproducible la configuración por I2C, la captura, la sincronización de fotogramas, la transmisión y el procesamiento de comandos.

ENTORNO DE HARDWARE REAL: Se dispone del sistema desplegado sobre la placa Basys 3, con la cámara MT9V111 conectada mediante la PCB adaptadora y el módulo FT232H comunicando con el PC. Es el entorno en el que se validan la integración física, el rendimiento real (fps), la detección de caras con imagen real y el funcionamiento de la aplicación *vicon_view*. La depuración sobre Hardware se apoya en el ILA de Vivado.

Las pruebas cuyo comportamiento puede reproducirse a nivel funcional se verifican primero en el entorno de simulación y, posteriormente, se confirman sobre hardware. Las pruebas que dependen de magnitudes físicas (rendimiento en fps, iluminación

real, integración de la PCB) o de la aplicación del Host(PC) se verifican únicamente sobre hardware.

9.1.2 Descripción de las pruebas

Tabla 9.1: Prueba P1.

Campo	Contenido
Requisito/s	R8, R9.1, R9.2, R2.1
Entorno	Hardware
Prueba	Con el sistema montado (cámara MT9V111 conectada a los conectores Pmod JA y JXADC mediante la PCB adaptadora), se programa el bitstream en la Basys 3 y se conecta el módulo FT232H al PC. Se ejecuta el SW en el Host y se observa el enlace. Es una prueba de integración física que solo puede realizarse sobre Hardware.
Salida	■ El bitstream se programa sin errores en la FPGA. ■ <code>vicon_view</code> abre el dispositivo FT232H y comienza a recibir datos. ■ El sensor entrega señal de imagen válida a través de la PCB adaptadora.
Errores	■ La FPGA no acepta el bitstream (recursos o pines mal asignados). ■ El PC no reconoce el FT232H o no recibe datos. ■ El sensor no entrega señal de imagen válida (conexión de la PCB defectuosa).

Tabla 9.2: Prueba P2.

Campo	Contenido
Requisito/s	R1.1, R8
Entorno	Simulación y hardware
Prueba	Se ejecutan las transacciones I2C de configuración de la cámara durante el arranque. En simulación se comprueban sobre el <i>testbench</i> de sistema (respuesta del modelo del sensor); en hardware se pueden observar con el ILA de Vivado o con un analizador lógico externo.
Salida	■ Las transacciones I2C de configuración se completan con ACK del sensor en simulación. ■ En hardware, el ILA muestra las mismas transacciones sobre el bus I2C real.
Errores	■ Ausencia de ACK del sensor (dirección o temporización incorrectas). ■ La secuencia de configuración difiere entre simulación y hardware.

Tabla 9.3: Prueba P₃.

Campo	Contenido
Requisito/s	R1.2, R2.1, R2.2, R5.1, R5.2
Entorno	Simulación y hardware
Prueba	Con la captura activada, se genera un fotograma y se recibe el flujo de vídeo. En simulación se estimula el diseño con la imagen sintética y se captura la salida a través del modelo del FT232H; en hardware se guarda un fotograma real (o la propia imagen sintética para comprobar píxeles de valor conocido) desde el SW . Se comprueban dimensiones, formato y la correcta delimitación por el marcador de inicio.

Salida

- Cada fotograma tiene una resolución de 640×480 píxeles y se representa en escala de grises.
- El marcador de inicio delimita correctamente cada imagen, tanto en simulación como en hardware.

Errores

- Los fotogramas aparecen desplazados o *rotos* (pérdida de sincronización del marcador). La imagen se "mueve" horizontal o verticalmente.
- Resolución distinta de 640×480 o aparición de componentes de color no esperadas.

Tabla 9.4: Prueba P4.

Campo	Contenido
Requisito/s	R4, R4.1, R4.2
Entorno	Simulación y hardware
Prueba	Se envían comandos desde el lado del PC (activar/desactivar captura y escritura de registro I2C). En simulación se inyectan a través del modelo bidireccional del FT232H en el <i>testbench</i> de sistema; en hardware se envían desde el SW y se verifican en la imagen y/o en el ILA.
Salida	■ La FPGA decodifica y ejecuta cada comando recibido. ■ La captura se activa/desactiva y la escritura de registro se aplica en la cámara, en simulación y hardware.
Errores	■ La FPGA no responde al comando o ejecuta una acción distinta. ■ El comando se pierde o se corrompe en el enlace (RX).

Tabla 9.5: Prueba P5.

Campo	Contenido
Requisito/s	R7
Entorno	Simulación y hardware
Prueba	Se verifica la comunicación con el FT232H en modo FIFO (Asíncrono como mínimo; Síncrono si es viable). En simulación, el modelo del FT232H comprueba el protocolo de lectura/escritura; en hardware se transmite vídeo de forma continuada comprobando la integridad del flujo.
Salida	■ El protocolo de transferencia con el FT232H funciona correctamente en el <i>testbench</i> de sistema. ■ En hardware, el SW recibe el flujo sin pérdida apreciable de fotogramas.
Errores	■ Fallo de protocolo (temporización de RD#/WR#/OE#) en simulación. ■ Pérdida o corrupción de fotogramas por desbordamiento del búfer.

Tabla 9.6: Prueba P6.

Campo	Contenido
Requisito/s	R5.3, R6
Entorno	Hardware
Prueba	Se mide por software la tasa de fotogramas recibidos durante 10 segundos en condiciones nominales de iluminación, observando la estabilidad del retardo de visualización. El rendimiento real solo es representativo sobre hardware.
Salida	■ La tasa media es igual o superior a 10 fps. ■ La visualización se mantiene continua, sin acumular retardo creciente.
Errores	■ Tasa inferior a 10 fps de forma sostenida. ■ El retardo entre captura y visualización crece con el tiempo y deja de ser tiempo real.

Tabla 9.7: Prueba P7.

Campo	Contenido
Requisito/s	R3.1, R3.2, R3.3
Entorno	Hardware
Prueba	Se ejecuta la aplicación del Host y se observa la ventana de visualización durante la recepción de vídeo.
Salida	■ La GUI muestra el vídeo en una ventana central en tiempo real. ■ La GUI muestra el valor de FPS actualizado y presenta los botones laterales de control.
Errores	■ La ventana no se actualiza o muestra la imagen congelada. ■ El valor de FPS no se corresponde con la tasa real.

Tabla 9.8: Prueba P8.

Campo	Contenido
Requisito/s	R3.4
Entorno	Hardware
Prueba	Con una cara presente en el campo de visión de la cámara, se activa la detección en la aplicación del Host. Requiere imagen real, por lo que solo se verifica en hardware.
Salida	■ La aplicación resalta la cara detectada sobre el vídeo mediante OpenCV.
Errores	■ No se detecta la cara en condiciones nominales. ■ Se producen falsos positivos de forma continua.

Tabla 9.9: Prueba P9.

Campo	Contenido
Requisito/s	R9.3, R11.1, R11.2
Entorno	Hardware / entorno de desarrollo
Prueba	Se compila el código de <code>vicon_view</code> (C++) y se sintetiza el diseño de la FPGA (VHDL), ejecutando la aplicación en el PC de trabajo.
Salida	■ El código C++ compila sin errores junto con OpenCV. ■ El diseño VHDL sintetiza correctamente en Vivado. ■ La aplicación se ejecuta sin errores en un PC con Windows 11 y 8 GB de RAM.
Errores	■ Errores de compilación o de síntesis. ■ La aplicación no arranca por falta de recursos o dependencias.

9.2 PLAN DE VALIDACIÓN

Mientras que la verificación (Sección 9.1) comprueba que el sistema cumple los requisitos especificados, la validación comprueba que el sistema (VICON) resuelve la necesidad para la que fue diseñado. Para ello, las pruebas de validación se plantean sobre los casos de uso completos (Capítulo 4), contemplando el sistema de extremo a extremo en condiciones de uso reales.

La Tabla 9.10 relaciona cada entregable del proyecto con las pruebas que lo validan.

Tabla 9.10: Entregables y pruebas de validación asociadas.

Nombre del entregable	Pruebas asociadas para su validación
Sistema de captura y transmisión sobre FPGA	V1, V2
Aplicación de visualización en el Host (C++ SW)	V2, V3
Control y configuración de la cámara por comandos	V4
Detección de caras (funcionalidad fundamental)	V5

9.2.1 Pruebas del plan de validación

A continuación se describe cada prueba de validación indicando su descripción, prerequisites, pasos, resultado esperado y resultado obtenido.

Tabla 9.11: Prueba de validación V1.

Validación de la captura y transmisión de vídeo (C1 + C2) V1
Descripción: Se valida que el sistema captura el vídeo de la cámara y lo transmite íntegro al ordenador, cubriendo los casos de uso C1 (capturar fotograma) y C2 (transmitir vídeo) de extremo a extremo. Prerrequisitos: Sistema montado sobre la Basys 3, cámara conectada por la PCB adaptadora, módulo FT232H enlazado al PC e iluminación nominal de la escena. Pasos: 1. Programar el bitstream en la FPGA y conectar el FT232H al PC. 2. Activar la captura desde vicon_view. 3. Observar la recepción continua de fotogramas durante varios segundos. 4. Guardar uno de los fotogramas recibidos y comprobar su resolución y formato. Resultado esperado: El sistema entrega un flujo continuo de fotogramas de 640×480 píxeles en escala de grises, correctamente delimitados, sin fotogramas rotos ni desplazados. Resultado obtenido: SUPERADA. Tal y como se recoge en la Sección 6.2.1 (implementación) y en las pruebas en hardware del Capítulo 8, vicon_view recibe y muestra de forma continua el flujo de vídeo en escala de grises a resolución VGA (640×480), sin fotogramas rotos ni desplazados. (Ver Figura A.1)

Tabla 9.12: Prueba de validación V2.

Validación de la visualización en tiempo real (C ₃)	V ₂
Descripción: Se valida que el usuario visualiza el vídeo en tiempo real en la aplicación del Host a la tasa requerida, cubriendo el caso de uso C ₃ (visualizar vídeo).	
Prerrequisitos: Prueba V ₁ superada. Aplicación SW en ejecución en un PC con Windows 11 y 8 GB de RAM.	
Pasos: 1. Ejecutar aplicación con el sistema transmitiendo vídeo. 2. Mover un objeto delante de la cámara y observar la respuesta en la ventana. 3. Comprobar el valor de FPS mostrado durante al menos 10 segundos.	
Resultado esperado: La ventana muestra el vídeo de forma fluida y sin retardo creciente, con una tasa igual o superior a 10 fps reflejada en el indicador de FPS.	
Resultado obtenido: SUPERADA. El sistema transmite a la tasa objetivo de 30 fps (c_MT9V111_TARGET_FPS), muy por encima del mínimo de 10 fps exigido, y la Figura A.1 (Apéndice A) muestra el indicador <i>Cam FPS</i> activo en la interfaz. (Ver Figura A.3)	

Tabla 9.13: Prueba de validación V3.

Validación de la interfaz gráfica (disposición y control)	V3
Descripción: Se valida que la interfaz gráfica presenta la disposición esperada (vídeo central y controles laterales) y resulta usable para el manejo del sistema.	
Prerrequisitos: Aplicación en ejecución y recibiendo vídeo.	
Pasos: 1. Abrir la aplicación y comprobar la disposición de la ventana. 2. Localizar el área de vídeo central y los botones laterales de control. 3. Interactuar con los controles disponibles.	
Resultado esperado: La GUI presenta el vídeo en la zona central y los controles en los laterales, y el usuario puede manejar el sistema desde ella sin ambigüedad.	
Resultado obtenido: SUPERADA. La Figura A.1 (Apéndice A) muestra la disposición final de vicon_view: ventana de vídeo central y panel de control lateral (cámara, detección de rostros, LEDs, configuración I2C y log de comandos), acorde a los requisitos R3.3/R3.1.	

Tabla 9.14: Prueba de validación V4.

Validación del control y configuración por comandos (C4)	V4
Descripción: Se valida que el usuario puede controlar y configurar el sistema mediante comandos (activar/desactivar la captura y escribir registros de la cámara por I2C), cubriendo el caso de uso C4 (enviar comando).	
Prerrequisitos: Sistema en funcionamiento y transmitiendo vídeo.	
Pasos: 1. Enviar desde la GUI el comando de desactivar y volver a activar la captura. 2. Enviar un comando de escritura de un registro de la cámara por I2C. 3. Observar el efecto en el vídeo (o mediante el ILA).	
Resultado esperado: El sistema responde a cada comando: la captura se detiene y reanuda según lo solicitado, y la escritura del registro produce el efecto esperado en la imagen o se confirma con el ILA.	
Resultado obtenido: SUPERADA. Tras la corrección del protocolo FT232H (Sección 6.2.4), los cuatro comandos (CAP, I2C, LED, SIM) funcionan con un único envío, tal y como recoge la Sección “Comandos de control” del Capítulo 8; la Figura A.2 (Apéndice A) muestra un envío de comando I2C desde la GUI. Nota: los comandos I2C solo se atienden una vez el sistema ha terminado su configuración inicial (ST_FINISH, véase la Sección 6.2.5).	

Tabla 9.15: Prueba de validación V5.

Validación de la detección de caras (C3 ampliado, deseable) V5
Descripción: Se valida la funcionalidad deseable de detección de caras sobre el vídeo recibido, como paso hacia el objetivo final del proyecto.
Prerrequisitos: Prueba V2 superada y detección activada en la aplicación del Host.
Pasos: 1. Situar una cara en el campo de visión de la cámara. 2. Activar la detección de caras en la GUI. 3. Comprobar que la cara se resalta sobre el vídeo.
Resultado esperado: La aplicación detecta y resalta la cara presente en la escena mediante OpenCV, en condiciones de iluminación nominales.
Resultado obtenido: SUPERADA. La Figura A.3 (Apéndice A) muestra la detección y el resaltado correcto de múltiples caras presentadas al sensor real mediante una fotografía en una tablet, confirmando el funcionamiento de la detección basada en OpenCV sobre imagen real del MT9V111.

CONCLUSIONES Y TRABAJO FUTURO

En este capítulo se recogen las conclusiones extraídas del desarrollo del proyecto **VICON**, valorando el grado de cumplimiento de los objetivos planteados en el Capítulo 1, y se proponen las líneas de trabajo futuro que permitirían extender sus capacidades.

10.1 CONCLUSIONES

En este trabajo se ha diseñado, implementado y verificado el sistema de adquisición y transmisión de imagen VICON sobre la FPGA Basys 3. Las principales contribuciones son:

- **Protocolo FT232H correcto:** el proceso de depuración con ILA reveló que la corrección del protocolo *Synchronous FIFO* requería: (a) separar los procesos TX y RX en flancos distintos (*falling* y *rising edge* respectivamente), (b) implementar la lectura en modo ráfaga continua con RD# mantenido bajo durante todos los bytes, y (c) respetar el pipeline natural de un ciclo del FT232H. Este aprendizaje constituye uno de los resultados más valiosos del proyecto.
- **Gestión correcta de CDC:** se han identificado y resuelto todos los cruces de dominio de reloj del diseño, aplicando sincronizadores de doble flip-flop con atributo ASYNC_REG, restricciones set_false_path y set_clock_groups, y logrando un cierre de timing con WNS = +1,565 ns.
- **Diseño RTL completo en VHDL:** se han desarrollado desde cero los módulos de control I2C del sensor MT9V111, captura de trama, control del FT232H en modo *Synchronous FIFO* y procesador de comandos, todos parametrizables mediante genéricos y paquetes de constantes.
- **Banco de pruebas UVVM:** se ha desarrollado un banco de pruebas estructurado con el logger de UVVM, incluyendo agentes de simulación para el sensor I2C y el FT232H, monitores de FSM y frame, y un sniffer del bus I2C. La simulación alcanza el estado ST_FINISH completando satisfactoriamente las 13 transacciones I2C de configuración del sensor.
- **Sistema funcional en hardware:** el sistema completo opera correctamente en hardware real, transmitiendo imagen en escala de grises a 640×480@30 fps y respondiendo a todos los comandos de control (LED, BCD, I2C, CAP, SIM) con un único envío.

10.2 TRABAJO FUTURO

Las líneas de trabajo futuro identificadas son:

- **Soporte de color:** el sensor MT9V111 entrega datos en formato YCbCr 4:2:2. La extensión del sistema a imagen en color requeriría modificar `frame_capture` para capturar todos los bytes (eliminando el descarte de crominancia), doblar el tamaño del buffer en `vicon_view` y añadir un conversor YCbCr→RGB en C++.
- **Mejora del framework de verificación:** separar el *test harness* del secuenciador de test mediante el genérico GC_TESTCASE, permitiendo ejecutar múltiples casos de prueba sobre el mismo banco sin duplicar la infraestructura. Completar la interfaz del agente FTDI para inyección de comandos desde el secuenciador.
- **Cobertura y trazabilidad:** añadir cobertura funcional (estados FSM alcanzados, comandos ejercitados) y matriz de trazabilidad requisitos↔test cases usando las facilidades de UVVM.
- **Procesamiento en la FPGA:** explorar la implementación de operaciones de procesamiento de imagen básicas directamente en la FPGA (filtros, detección de bordes) aprovechando el paralelismo de la arquitectura y reduciendo la carga de transferencia hacia el PC.
- **Iluminación auxiliar para entornos de baja luminosidad:** el requisito R5.3 condiciona explícitamente la tasa de fotogramas a disponer de *condiciones de luz suficientes*", y la fiabilidad de la detección de caras (R3.4) se degrada notablemente en escenas poco iluminadas. Una posible mejora consiste en incorporar un anillo de LEDs (blancos o IR, según queramos más calidad visual o más discreción) alrededor del sensor MT9V111, controlado desde la FPGA a través de un driver/MOSFET externo (la Basys 3 no puede alimentar directamente LEDs de potencia desde sus GPIO). Esta mejora sería complementaria al control de exposición automático, no un sustituto de él.

BIBLIOGRAFÍA

- [1] FTDI Ltd. *FT232H Single Channel Hi Speed USB to Multipurpose UART/FIFO IC*. FTDI Ltd., 2015. URL https://ftdichip.com/wp-content/uploads/2020/07/DS_FT232H.pdf. Datasheet Version 2.6.
- [2] Aptina Imaging Corporation. *MT9V111: 1/4-Inch SOC VGA CMOS Active-Pixel Digital Image Sensor*. Aptina Imaging Corporation, 2004. URL <https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/992/MT9V111.pdf>. MT9V111 Datasheet, Rev. K.
- [3] Accellera Systems Initiative. *IEEE Standard for Universal Verification Methodology (UVM) Language Reference Manual*. IEEE / Accellera Systems Initiative, 2020. URL <https://accellera.org/downloads/standards/uvm>. IEEE Std 1800.2-2020.
- [4] OSVVM. *OSVVM: Open Source VHDL Verification Methodology*. OSVVM.org, 2023. URL <https://osvvm.org>. Documentación y librerías disponibles en el repositorio oficial.
- [5] cocotb contributors. *cocotb: A Coroutine Based Cosimulation Library for Writing VHDL and Verilog Testbenches in Python*. cocotb, 2024. URL <https://www.cocotb.org>. Documentación oficial.
- [6] Bitvis AS. *UVVM — Universal VHDL Verification Methodology*, 2023. Retrieved from: <https://github.com/UVVM/UVVM>.
- [7] Digilent Inc. *Basys 3 FPGA Board Reference Manual*. Digilent Inc., 2016. URL <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>. Basys 3 Reference Manual.
- [8] FTDI Ltd. *AN_130: FT232H Used In FT245 Synchronous FIFO Mode*. FTDI Ltd., 2010. URL https://ftdichip.com/wp-content/uploads/2020/08/AN_130_FT232H_Used_In_FT245-Synchronous-FIFO-Mode.pdf. Application Note AN_130.

Parte I

APENDICES

En esta parte se incluyen los manuales de usuario y documentación adicional.



MANUAL DE USUARIO

En este apéndice se detalla el manual de usuario correspondiente al sistema desarrollado, guiando paso a paso en el proceso de uso de la plataforma. Se asume que la **FPGA** ya tiene el diseño programado en su memoria de configuración; este manual cubre por tanto la puesta en marcha del hardware y el uso del software de PC.

A.1 REQUISITOS DEL SISTEMA

Antes de iniciar la instalación, hay que cumplir con los siguientes requisitos previos en su equipo:

- **Sistema operativo:** Windows 11, con un mínimo de 8 GB de RAM.
- **Basys 3:** Basys 3 con el *bitstream* de VICON ya programado en su memoria de configuración, conectada al sensor MT9V111 y al chip FT232H (módulo UM232H-B) utilizando los conectores *Pmod* indicados en la Figura 5.3.
- **Driver D2XX:** controlador D2XX de FTDI instalado en el PC (disponible en <https://ftdichip.com/drivers/d2xx-drivers/>), necesario para que la aplicación de PC acceda al FT232H.
- **Cable USB:** un cable USB 2.0 que conecte el módulo UM232H-B directamente a un puerto del PC. Se recomienda evitar conectarlo a través de un *hub* USB, ya que puede introducir latencias adicionales en la recepción del vídeo.

Antes de encender o alimentar la placa, comprobar la posición del jumper de modo de arranque (JP1). Para que la Basys 3 cargue el diseño desde su memoria de configuración no volátil, dicho jumper debe situarse en la posición QSPI; en la posición USB la placa esperaría en su lugar una programación por JTAG desde Vivado.

A.2 INSTALACIÓN Y CONFIGURACIÓN

Se deben seguir los pasos descritos a continuación para preparar el software de visualización:

A.2.1 Descarga del código fuente

Clonar el repositorio oficial del proyecto desde <https://github.com/carlosmgj/VICON>

A.2.2 Configuración del módulo FT232H

El módulo UM232H-B debe tener su [EEPROM](#) configurada en modo **FT245 FIFO** para poder operar en modo *Synchronous FIFO*. Esta configuración solo es necesaria una vez por dispositivo:

1. Instalar la utilidad **FT_Prog** de FTDI (requiere el driver D2XX ya instalado).
2. Conectar el UM232H-B al PC y, en FT_Prog, pulsar *Scan and Parse* para detectar el dispositivo.
3. Navegar hasta FT EEPROM → Hardware Specific → Port A → Hardware y cambiar el modo de UART a 245 FIFO.
4. Programar la [EEPROM](#) (icono del rayo) y, al finalizar, desconectar y vuelva a conectar el módulo para que el sistema operativo lo reconozca con la nueva configuración.

A.2.3 Compilación de la aplicación de PC

La aplicación `vicon_view` se distribuye como un ejecutable y por tanto no hace falta tener las dependencias instaladas. Únicamente serían necesarias para la compilación de la aplicación. La ruta del ejecutable es: `VICON/07-Versions/3.0/GUI/vicon_gui.exe`

A.3 GUÍA DE USO DE LA APLICACIÓN

A.3.1 Inicio de la aplicación

Con la Basys 3 encendida, el sensor conectado y el módulo FT232H enlazado al PC por USB, ejecutar `vicon_view`. La aplicación abre el dispositivo FT232H a través del *driver* D2XX y comienza a recibir el flujo de vídeo sintético de forma automática; no es necesario ningún paso adicional de emparejamiento o configuración del enlace.

A.3.2 Descripción de la interfaz

Una vez iniciada la interfaz de usuario, se presentan dos zonas principales:

1. **Ventana de vídeo:** muestra en tiempo real el fotograma recibido desde la [FPGA](#) en escala de grises, junto con la tasa de fotogramas por segundo ([FPS](#)) actual.
2. **Panel de control:** Se encuentran los controles para enviar comandos a la [FPGA](#), descritos en la Tabla [A.1](#).

Por defecto, la interfaz muestra el video sintético generado para validar la comunicación entre el sensor y el Host a través de la FPGA como se ve en la Figura [A.1](#):

Para configurar los FPS de la imagen a mínimo 30fps, se deberá configurar el registro I2C correspondiente mandando la trama `1 0x37 0x0080` (Ver Figura [A.2](#)). Este paso se puede realizar antes o después de cambiar el modo de la imagen.

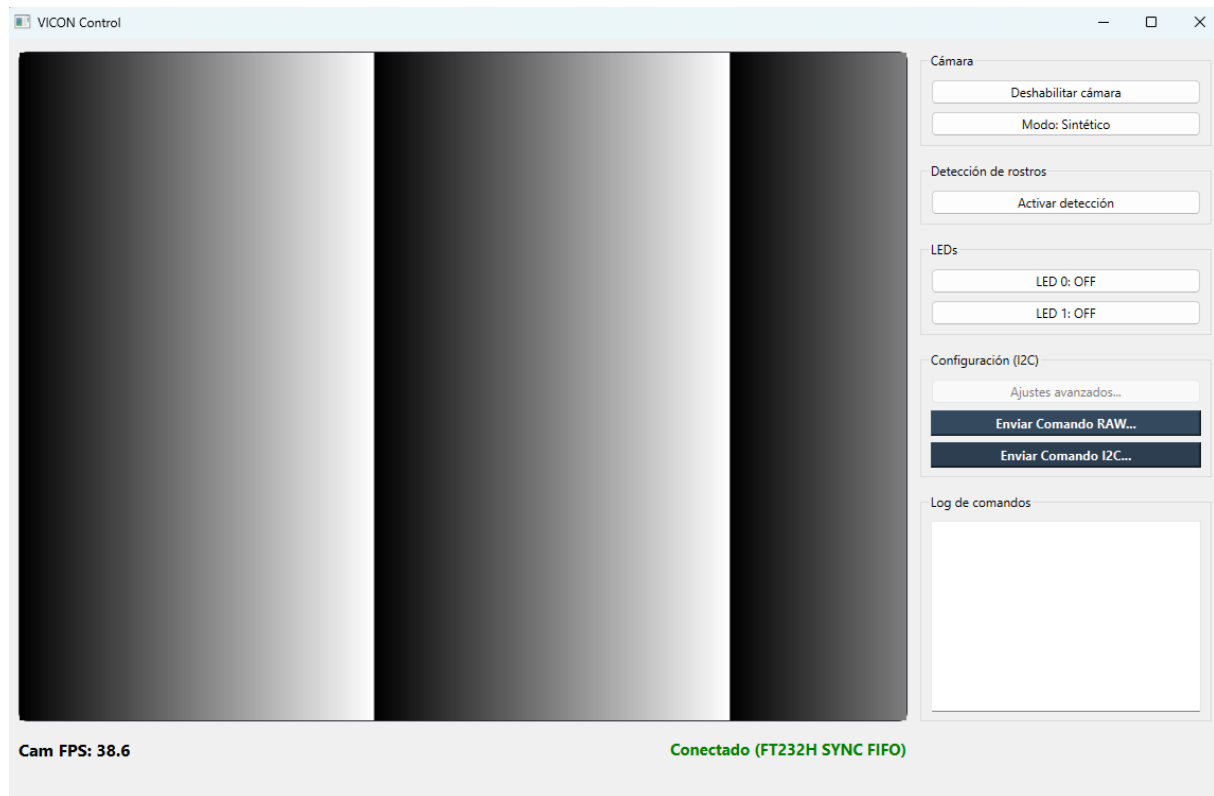


Figura A.1: Estado Inicial de la Interfaz Gráfica de Usuario.

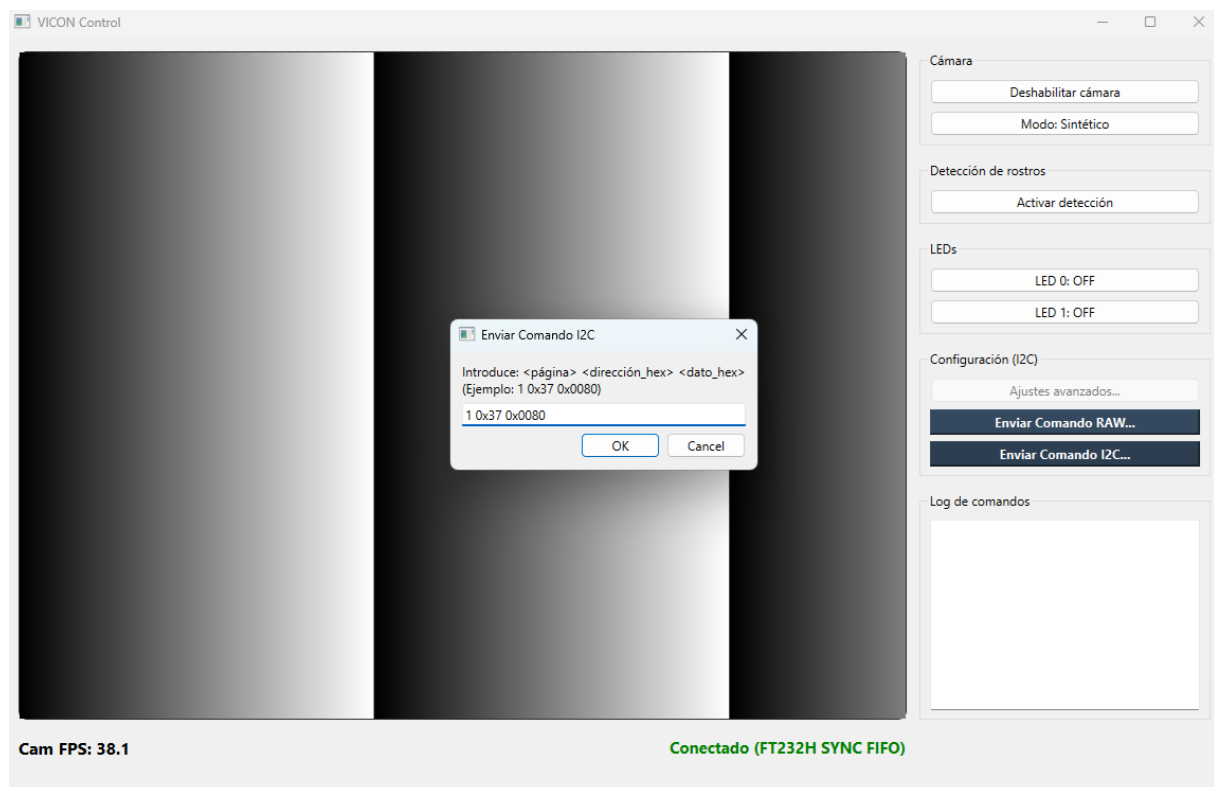


Figura A.2: GUI: envío de comando I2C para configurar >30fps.

Si a continuación habilitamos la imagen del sensor real y la detección de rostros, el software remarcará la/s cara/s que detecte en la imagen. En la Figura A.3 se puede ver el resultado de presentar al sensor una foto en una tablet con múltiples caras.

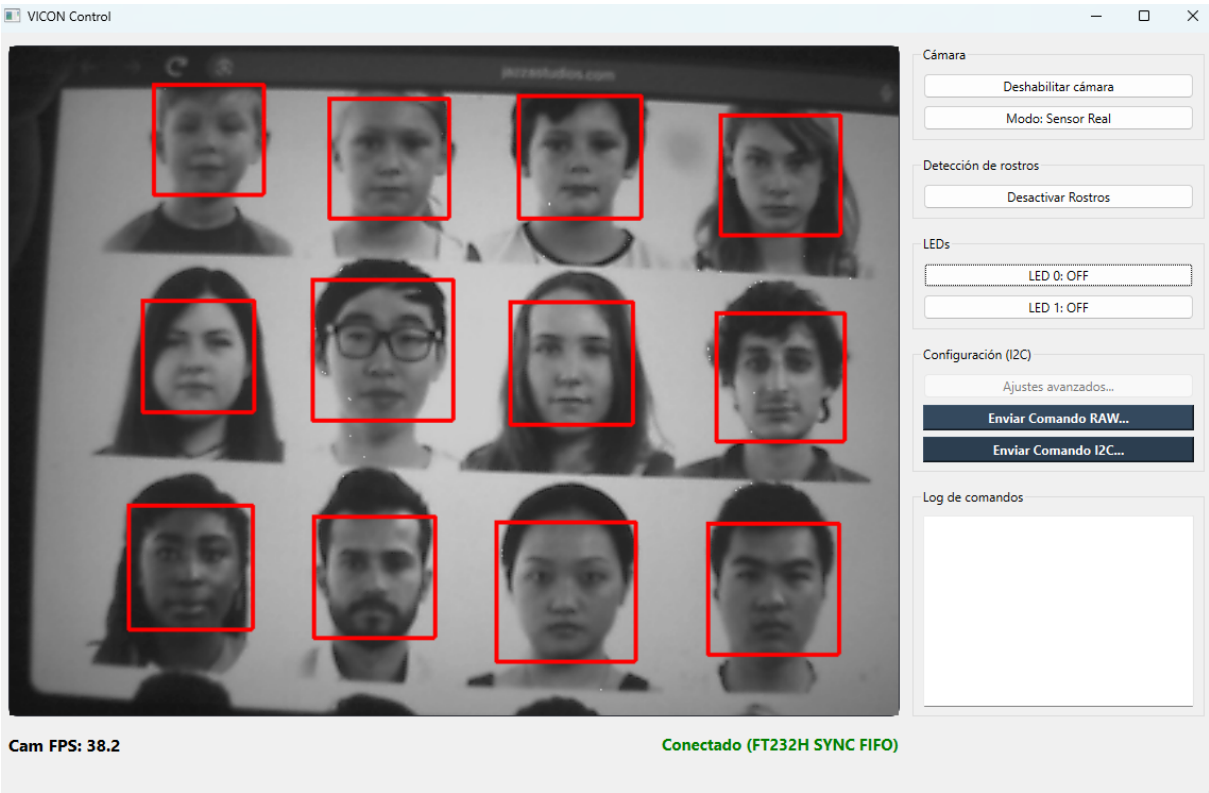


Figura A.3: GUI: envío de comando I2C para configurar >30fps.

Tabla A.1: Comandos disponibles desde la interfaz de vicon_view.

Comando	Efecto
LED	Conmuta el estado de un LED de la Basys 3, útil como comprobación rápida de que el canal de comandos PC→FPGA funciona.
Activar/desactivar captura	Detiene o reanuda la transmisión de vídeo desde la FPGA sin necesidad de desconectar el enlace USB.
Imagen sintética	Conmuta entre el sensor MT9V111 y el generador interno de imagen sintética, útil para validar el resto de la cadena sin depender de la cámara física.
Escritura de registro I2C	Permite escribir un valor en un registro concreto del sensor (página, dirección y dato), para ajustar su configuración en caliente.

A.4 RESOLUCIÓN DE PROBLEMAS (*troubleshooting*)

En la Tabla A.2 se resumen los errores más habituales y las acciones recomendadas para su resolución.

Tabla A.2: Problemas frecuentes y soluciones.

Error	Causa probable	Solución recomendada
No se ven los LEDs encendidos al arrancar la placa.	<i>Jumper</i> de modo mal colocado.	Situar el <i>jumper</i> JP1 en modo QSPI antes de alimentar la placa.
La aplicación no detecta el FT232H.	Windows ha instalado automáticamente el <i>driver</i> VCP (puerto serie virtual) en lugar del D2XX.	Reinstalar el <i>driver</i> D2XX desde el Administrador de dispositivos, seleccionando manualmente la carpeta de instalación de FTDI.
FT_Prog no reconoce el dispositivo al configurar la EEPROM.	Conflicto entre el <i>driver</i> D2XX y otro <i>driver</i> (p. ej. libusb) instalado previamente sobre el mismo dispositivo.	Reinstalar el <i>driver</i> D2XX y volver a conectar el cable antes de reintentar con FT_Prog.
El vídeo se congela o aparecen fotogramas corruptos de forma intermitente.	Sobreescritura de la FIFO interna por una latencia de lectura excesiva en el PC (agravada si el FT232H está conectado a través de un <i>hub</i> USB).	Conectar el módulo directamente a un puerto USB del equipo, sin <i>hubs</i> intermedios.

DIAGRAMAS DE ESTADOS (FSM)

Este anexo recoge los diagramas de estados completos de las máquinas de estados finitas (FSM) más voluminosas del diseño, referenciadas desde sus respectivos módulos en el Capítulo 6. Se han trasladado aquí, en lugar de incluirse en el cuerpo del documento, debido a su tamaño.

B.1 FSM DE CONFIGURACIÓN DEL TOP

Diagrama de estados de la FSM principal `p_fsm` (`TOP.vhd`), descrita textualmente en la Sección 6.2.1.

B.2 FSM DEL `i2c_controller`

Diagrama de estados completo del maestro `I2C` `i2c_controller.vhd` (37 estados, Tabla 6.1, Sección 6.2.2).

B.3 FSM DE RECEPCIÓN DEL `ftdi_controller`

Diagrama de estados del proceso `p_rx` de `ftdi_controller.vhd` (Sección 6.2.4), que gestiona la recepción de comandos PC→FPGA en modo ráfaga continua.

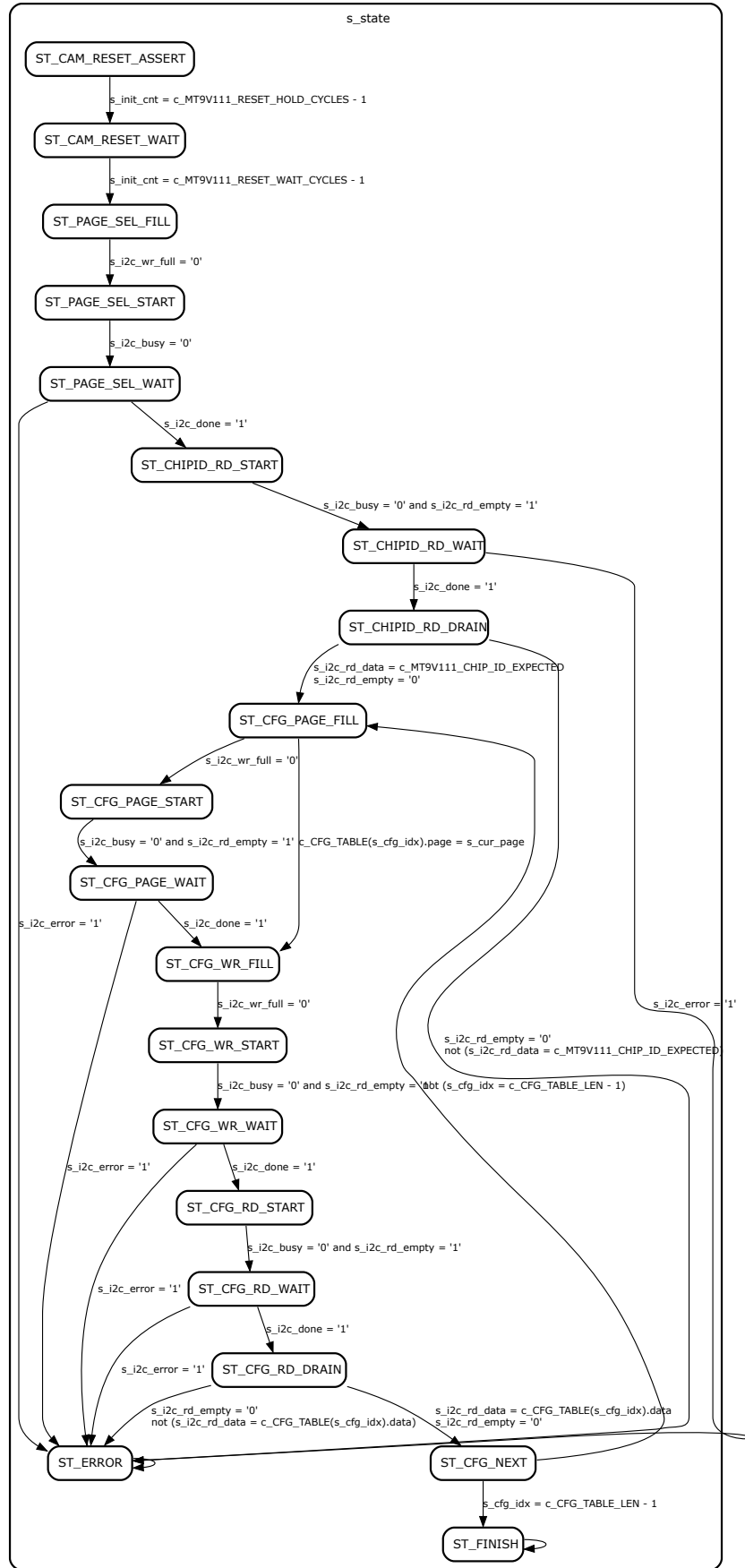
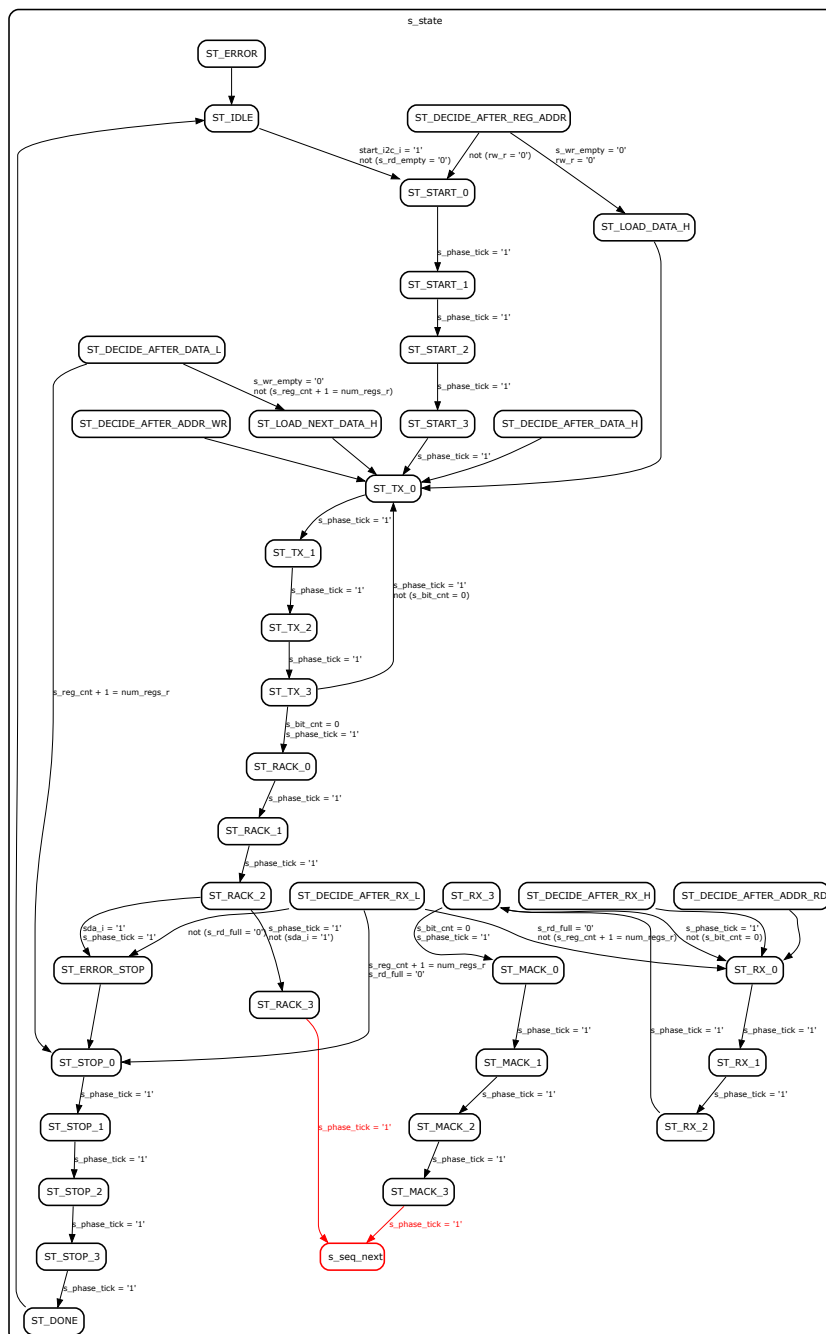


Figura B.1: FSM de configuración del sensor en TOP.vhd.

Figura B.2: FSM de `i2c_controller.vhd`.

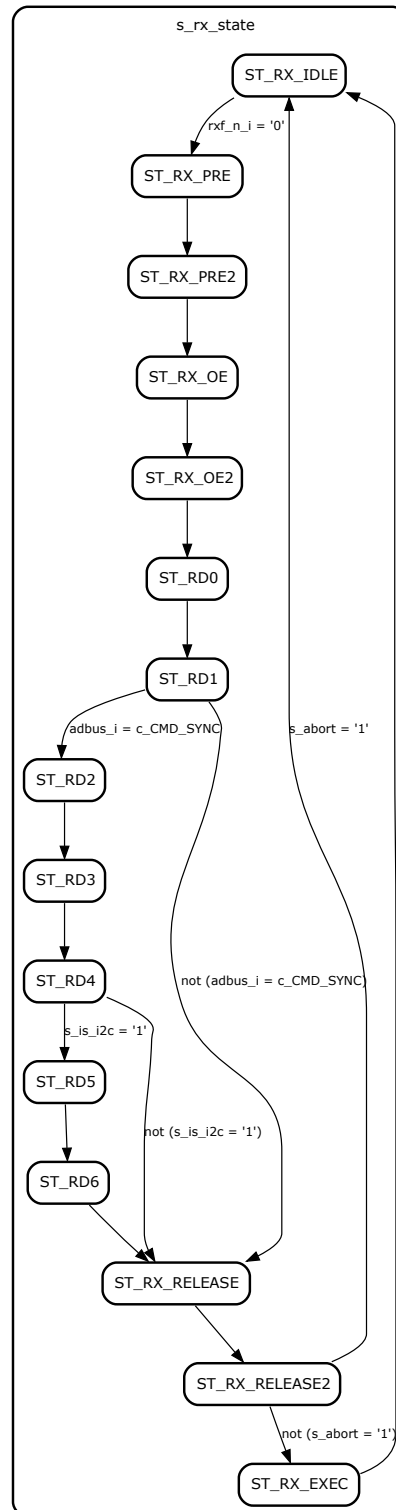


Figura B.3: FSM de recepción (p_rx) de ftdi_controller.vhd.